

containerization-and-devops

sap id : 500119144 Name : Vaani Jindal

Lab Experiments

 [Experiment 1](#)

 [Experiment 2](#)

 [Experiment 3](#)

 [Experiment 4](#)

 [Experiment 5](#)

 [Experiment 6](#)

 [Experiment 7](#)

 [Experiment 9](#)

 [Experiment 10](#)

 [Experiment 11](#)

 [C-D Theory \(README\):](#)

 [View C-D Theory Class Tasks](#)

containerization-and-devops

Experiment 1: Comparison of Virtual Machines and Containers

Student Details [🔗](#)

- Name: Vaani Jindal
- SAP ID: 500119144
- Batch: 2 CCVT

Aim

To compare **Virtual Machines** and **Containers** by deploying **Nginx** on a Virtual Machine using **Vagrant** and inside a **Docker container**, and observing their resource utilization.

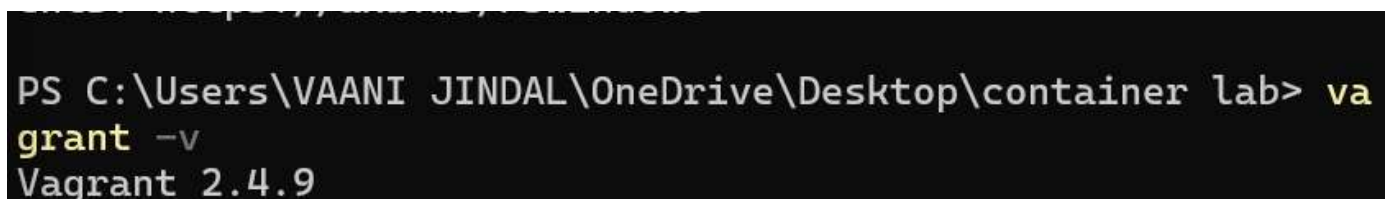
Part A: Virtual Machine Setup (Windows)

Step 1: Install VirtualBox

Install VirtualBox on your system to create and manage virtual machines.

Step 2: Install Vagrant

Install Vagrant to manage and configure virtual machines easily.
Verify that Vagrant is installed properly after installation.

A terminal window with a black background and white text. The prompt is 'PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab>'. The user has entered 'vagrant -v' and the output is 'Vagrant 2.4.9'.

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> va
grant -v
Vagrant 2.4.9
```

Step 3: Initialize Vagrant with Ubuntu Box

Initialize Vagrant with an Ubuntu box.

Start the virtual machine and access it using SSH.

```
vagrant 2.4.7
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> va
grant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are
now
ready to 'vagrant up' your first virtual environment! Pleas
e read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
```

```
default: C:/Users/VAANI JINDAL/OneDrive/Desktop/container lab -> /vagrant
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

System information as of Sat Jan 24 03:18:39 UTC 2026

```
System load:  0.13           Processes:            91
Usage of /:   2.5% of 61.80GB Users logged in:       0
Memory usage: 11%           IP address for eth0: 10.0.2.15
Swap usage:   0%
```

```
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.
```

```
vagrant@vagrant:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [102 kB]
Hit:2 http://archive.ubuntu.com/ubuntu bionic InRelease
Get:3 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [102 kB]
Get:4 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [102 kB]
Get:5 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [2,717 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [3,045 kB]
Get:7 http://archive.ubuntu.com/ubuntu bionic-updates/main i386 Packages [1,666 kB]
Get:8 http://security.ubuntu.com/ubuntu bionic-security/main i386 Packages [1,380 kB]
Get:9 http://archive.ubuntu.com/ubuntu bionic-updates/main Translation-en [554 kB]
Get:10 http://security.ubuntu.com/ubuntu bionic-security/main Translation-en [468 kB]
Get:11 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Packages [1,317 kB]
Get:12 http://archive.ubuntu.com/ubuntu bionic-updates/restricted i386 Packages [39.7 kB]
Get:13 http://archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [1,347 kB]
Get:14 http://security.ubuntu.com/ubuntu bionic-security/restricted i386 Packages [33.0 kB]
Get:15 http://security.ubuntu.com/ubuntu bionic-security/restricted Translation-en [182 kB]
```

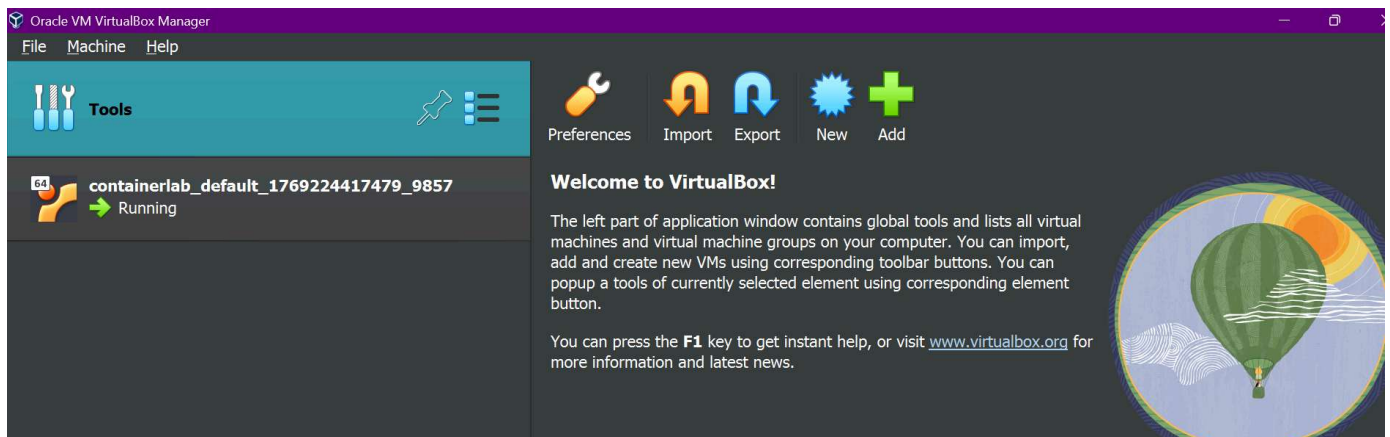
Step 4: Install Nginx inside Virtual Machine

Update the system and install Nginx inside the virtual machine.

Start the Nginx service after installation.

```
vagrant@vagrant:~$ sudo apt install -y nginx
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjbig0 libjpeg-turbo8 libjpeg8
  libnginx-mod-http-geoip libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter
  libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6 libx11-6 libx11-data libxcb1 libxpm4
  nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjbig0 libjpeg-turbo8 libjpeg8
  libnginx-mod-http-geoip libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter
  libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6 libx11-6 libx11-data libxcb1 libxpm4 nginx
  nginx-common nginx-core
0 upgraded, 21 newly installed, 0 to remove and 294 not upgraded.
```

```
vagrant@vagrant:~$ sudo systemctl start nginx
```



Step 5: Verify Nginx in Virtual Machine

Check whether the Nginx server is running properly in the virtual machine.

```
vagrant@vagrant:~$ exit
logout
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> vagrant halt
==> default: Attempting graceful shutdown of VM...
```

Stop and Remove Virtual Machine

Stop the running virtual machine and remove it when it is no longer needed.

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> vagrant destroy
default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> |
```

```
vagrant@vagrant:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	985M	66M	453M	4.9M	465M	777M
Swap:	979M	0B	979M			

Resource Utilization Observation (Virtual Machine)

Observe the resource usage such as memory, CPU, and system performance inside the virtual machine.

Experiment Setup – Part B: Containers using WSL (Windows)

Step 1: Install WSL

Install Windows Subsystem for Linux (WSL) on your system.

```
Snapshot: 37511 KB
vagrant@vagrant:~$ htop
vagrant@vagrant:~$ l:systemd-analyze
-bash: l:systemd-analyze: command not found
vagrant@vagrant:~$ system-analyze
-bash: system-analyze: command not found
vagrant@vagrant:~$ systemd-analyze
Startup finished in 10.223s (kernel) + 1min 10.392s (userspace) = 1min 20.616s
graphical.target reached after 14.670s in userspace
```

Step 2: Install Ubuntu on WSL

Install Ubuntu distribution using WSL.

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\container lab> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab$ sudo apt update
[sudo] password for vaani:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
```

Step 3: Install Docker Engine inside WSL

Install Docker inside the WSL environment.

Start the Docker service and configure user permissions.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab$ sudo systemctl sta
rt docker
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab$ sudo usermod -aG d
ocker $USER
```

Step 4: Run Ubuntu Container with Nginx

Run an Ubuntu container and deploy Nginx inside it.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab$ sudo docker run -d
-p 80
--name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfe0ee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
e2776e5b3dc5f97bd6022403b34522fec2f8de0da8cb4796afe22d095691ea5
```

Step 5: Verify Nginx in Container

Verify that the Nginx server is working properly inside the container.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
CONTAINER ID   NAME                CPU %      MEM USAGE / LIMIT   MEM %      NET I/O       BLOCK
I/O          PIDS
e2776e5b3dc5   nginx-container     0.00%      7.629MiB / 7.641GiB  0.10%      2.09kB / 1.4kB   0B / 1
2.3kB       9
```

Step 6: Container Observation

Monitor the resource utilization of the running container.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab$ free -h
Mem:           7.6Gi       555Mi       7.0Gi       3.6Mi       243Mi       7.1Gi
Swap:          2.0Gi         0B         2.0Gi
```

Comparison: Virtual Machine vs Container

Feature	Virtual Machine	Container
Boot Time	Slow	Fast
Resource Usage	High	Low
Isolation	Full System	Process Level
Performance	Moderate	High

Feature	Virtual Machine	Container
Size	Large	Small

Conclusion

In this experiment, Virtual Machines and Containers were compared based on their setup, performance, and resource utilization.

Virtual Machines provide complete system isolation but consume more resources.

Containers are lightweight, faster, and efficient for application deployment.

Therefore, containers are more suitable for modern cloud and DevOps environments.

containerization-and-devops

Experiment 2: Docker Image and Container Lifecycle

Student Details

- **Name:** Vaani Jindal
- **SAP ID:** 500119144
- **Batch:** 2 CCVT

Objective

To pull Docker images, run containers, and perform basic container lifecycle operations.

Experiment Setup – Part B: Docker Container Management

Step 1: Pull Docker Image

Download the Nginx Docker image from Docker Hub to the local system.

Step 2: Run Container with Port Mapping

Create and run a Docker container using the Nginx image with proper port mapping.

Step 3: Verify Running Containers

Check the list of active Docker containers to confirm that the container is running.

Step 4: Stop and Remove Container

Stop the running container and remove it from the system.

Step 5: Remove Docker Image

Delete the Nginx Docker image from the local system.

Step 6: Re-run Container After Image Deletion

After deleting the image, try running the container again to check whether Docker downloads the image automatically.

Step 7: Verify Container on Localhost

Check the localhost port to verify that the Nginx server is running properly.

```
PS C:\Users\VAANI JINDAL> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d -p 8080:80 nginx
7eeab66a7c941186bee0f3a1c4c47f36cb8a527a9c215cc16e5e31aacf68f02b
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
7eeab66a7c94   nginx     "/docker-entrypoint..." 13 seconds ago Up 12 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   admiring_gagarin
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker stop 7eeab66a7c94
7eeab66a7c94
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
7eeab66a7c94   nginx     "/docker-entrypoint..." 20 hours ago   Exited (0)    16 seconds ago                    admiring_gagarin
b01d49f91071   java-app:1.0 "bash"                20 hours ago   Exited (0)    20 hours ago                    brave_kilby
a0a1c62b7af0   java-app:1.0 "java Hello"          20 hours ago   Exited (0)    20 hours ago                    quizzical_allen
5eccb98fe5fc   java-app:1.0 "bash"                2 days ago     Exited (129)  2 days ago                    unruffled_goodall
7a0aa0201bad   java-app:1.0 "java Hello"          2 days ago     Exited (0)    2 days ago                    eager_vaughan
5274c28b240b   myrepo/java-app:1.0 "bash"            3 days ago     Exited (127)  3 days ago                    stupefied_bardeen
820234c1af82   ubuntu:22.04    "bash"            3 days ago     Exited (129)  3 days ago                    java_lab
e2776e5b3dc5   nginx          "/docker-entrypoint..." 6 days ago     Exited (0)    6 days ago                    nginx-container
4f13e54fd5ea   ubuntu        "bash"            7 days ago     Exited (255)  2 days ago                    javac
b77e77dab633   ubuntu        "/bin/bash"        8 days ago     Exited (255)  7 days ago                    vigorous_easley
9beb34e25a52   ubuntu        "/bin/bash"        8 days ago     Exited (0)    8 days ago                    jovial_kare
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm 7eeab66a7c94
7eeab66a7c94
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
b01d49f91071   java-app:1.0 "bash"                20 hours ago   Exited (0)    20 hours ago                    brave_kilby
a0a1c62b7af0   java-app:1.0 "java Hello"          20 hours ago   Exited (0)    20 hours ago                    quizzical_allen
5eccb98fe5fc   java-app:1.0 "bash"                2 days ago     Exited (129)  2 days ago                    unruffled_goodall
7a0aa0201bad   java-app:1.0 "java Hello"          2 days ago     Exited (0)    2 days ago                    eager_vaughan
5274c28b240b   myrepo/java-app:1.0 "bash"            3 days ago     Exited (127)  3 days ago                    stupefied_bardeen
820234c1af82   ubuntu:22.04    "bash"            3 days ago     Exited (129)  3 days ago                    java_lab
e2776e5b3dc5   nginx          "/docker-entrypoint..." 6 days ago     Exited (0)    6 days ago                    nginx-container
4f13e54fd5ea   ubuntu        "bash"            7 days ago     Exited (255)  2 days ago                    javac
b77e77dab633   ubuntu        "/bin/bash"        8 days ago     Exited (255)  7 days ago                    vigorous_easley
9beb34e25a52   ubuntu        "/bin/bash"        8 days ago     Exited (0)    8 days ago                    jovial_kare
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo docker rmi nginx -f
Untagged: nginx:latest
Untagged: nginx@sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Deleted: sha256:4af177a024eb8a1e43f4fb6c66735bb8260115cb5925a64f51673219bd97c144
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |
```

Observation

- The Docker image was successfully pulled.
- The container was created and executed properly.
- Port mapping allowed access through the browser.
- After deleting the image, Docker downloaded it again automatically.
- Container lifecycle commands worked as expected.

Conclusion

In this experiment, Docker images were pulled successfully and containers were executed.

Basic lifecycle operations such as starting, stopping, removing containers, and deleting images were performed.

This experiment helped in understanding how Docker manages containers and images efficiently.

containerization-and-devops

Experiment 3: Deploying Nginx Using Docker

Objective [🔗](#)

To deploy and verify Nginx using:

- Official Docker Image
- Custom Ubuntu Base Image
- Custom Alpine Base Image

◆ Part 1: Deploy Nginx Using Official Image

Step 1: Pull the Official Nginx Image

Download the official Nginx image from Docker Hub.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo apt update && sudo apt upgrade -y
[sudo] password for vaani:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1739 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1528 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:15 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7296 B]
Get:16 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 4322 kB in 6s (733 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
127 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
Processing triggers for nvidia-driver-installer (535.177.01) ...
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (28.2.2-0ubuntu1~24.04.1).
The following packages were automatically installed and are no longer required:
  libllvm19 libwayland-server0
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```



```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo dockerd
INFO[2026-02-07T03:41:45.499607956Z] Starting up
WARN[2026-02-07T03:41:45.501727434Z] Binding to IP address without --tlsverify is insecure and gives root access on this machine to everyone who has access to your network. host="tcp://0.0.0.0:2375"
WARN[2026-02-07T03:41:45.501815171Z] Binding to an IP address, even on localhost, can also give access to scripts run in a browser. Be safe out there! host="tcp://0.0.0.0:2375"
WARN[2026-02-07T03:41:46.502938111Z] [DEPRECATION NOTICE] In future versions this will be a hard failure preventing the daemon from starting! Learn more at: https://docs.docker.com/go/api-security/ host="tcp://0.0.0.0:2375"
WARN[2026-02-07T03:41:46.503096842Z] Binding to an IP address without --tlsverify is deprecated. Startup is intentionally being slowed down to show this message host="tcp://0.0.0.0:2375"
WARN[2026-02-07T03:41:46.503096842Z] Please consider generating TLS certificates with client validation to prevent exposing unauthenticated root access to your network host="tcp://0.0.0.0:2375"
WARN[2026-02-07T03:41:46.503128271Z] You can override this by explicitly specifying '--tls=false' or '--tlsverify=false' host="tcp://0.0.0.0:2375"
WARN[2026-02-07T03:41:46.503198695Z] Support for listening on TCP without authentication or explicit intent to run without authentication will be removed in the next release host="tcp://0.0.0.0:2375"
INFO[2026-02-07T03:42:02.499657665Z] OTEL tracing is not configured, using no-op tracer provider
INFO[2026-02-07T03:42:02.639991375Z] Creating a containerd client address=/run/containerd/containerd.sock timeout=1m0s
INFO[2026-02-07T03:42:02.711409805Z] [graphdriver] using prior storage driver: overlay2
INFO[2026-02-07T03:42:02.819196912Z] Loading containers: start.
WARN[2026-02-07T03:42:03.631005806Z] Error (Unable to complete atomic operation, key modified) deleting object [endpoint_count 505fc1c2d6de2bfb2f542160ccf3473edfc608a15be7d1a7021ea6088f2e9be8], retrying...
WARN[2026-02-07T03:42:03.726765304Z] error locating sandbox id d376542a82a3f0ad70b934b85a537f21d0ffe42b41b319b268591f60fb9e8142: sandbox d376542a82a3f0ad70b934b85a537f21d0ffe42b41b319b268591f60fb9e8142 not found
WARN[2026-02-07T03:42:03.726858852Z] error locating sandbox id 5645da58bdca279789976600b917186d9cb765bbaf9db3437ba4018723aba0bd: sandbox 5645da58bdca279789976600b917186d9cb765bbaf9db3437ba4018723aba0bd not found
INFO[2026-02-07T03:42:03.727390216Z] Loading containers: done.
WARN[2026-02-07T03:42:03.765933287Z] [DEPRECATION NOTICE]: API is accessible on http://0.0.0.0:2375 without encryption. Access to the remote API is equivalent to root access on the host. Refer to the 'Docker daemon attack surface' section in the documentation for more information: https://docs.docker.com/go/attack-surface/

```

```

PS C:\Users\VAANI JINDAL> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker pull nginx:latest
latest: Pulling from library/nginx
0c8d55a45c0d: Already exists
46bf3a120c8e: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
f73400a233fd: Pull complete
47cd406a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

```

Step 2: Run the Container

Start a container using the downloaded image.

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d --name nginx-official -p 8080:80 nginx
0b25dfccdd9b7fafa5dbd4e6ea98860886b5aba01611f6dab0975fcc84f06994

```

Step 3: Verify Deployment

Open the browser and check whether the Nginx welcome page is displayed.


```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Step 4: Check Container Status

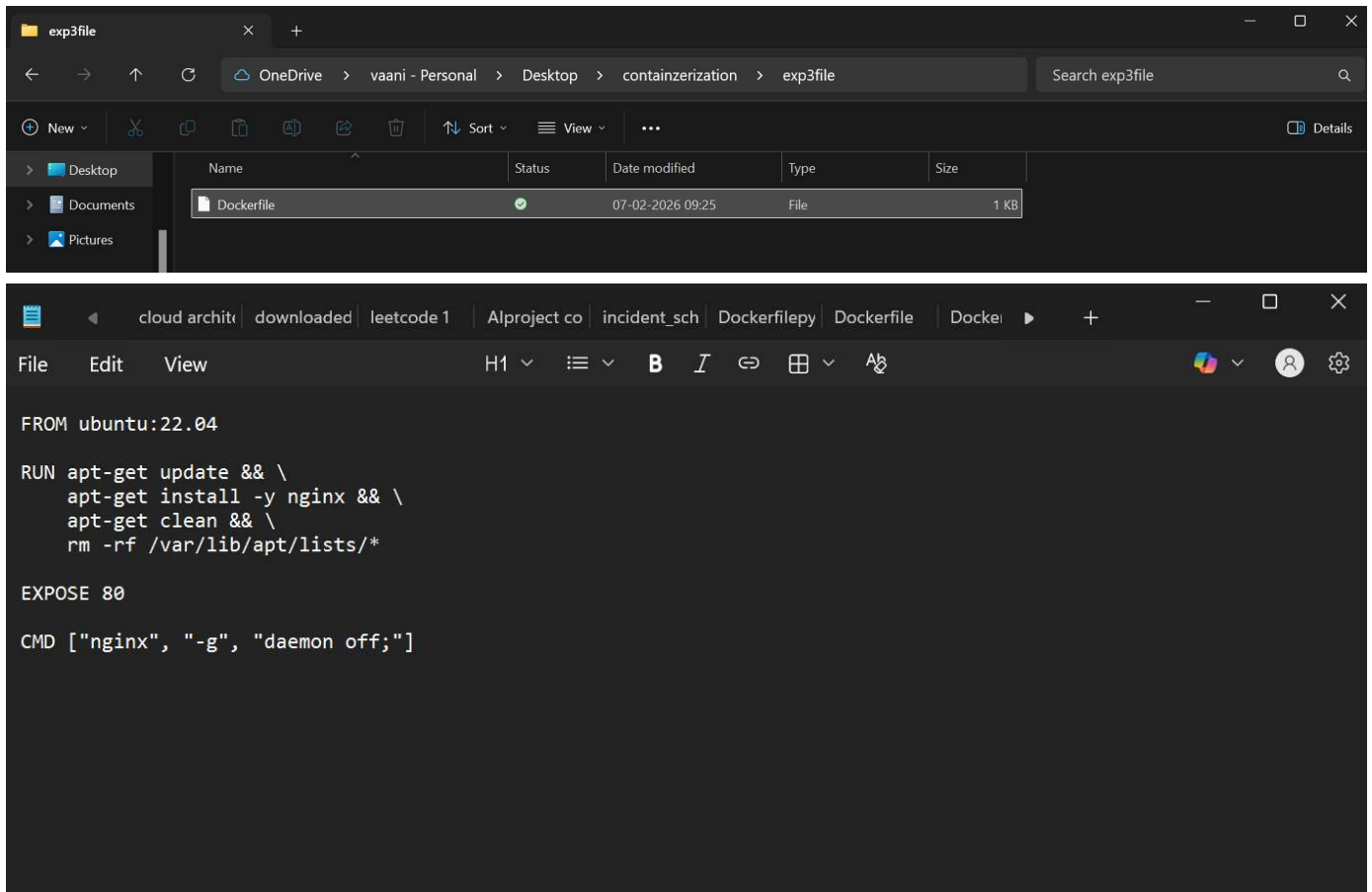
Verify that the container is running successfully.

```
</html>
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker images nginx
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    5cdef4ac3335   2 days ago    161MB
nginx         <none>    248d2326f351   4 days ago    161MB
nginx         <none>    4af177a024eb   3 weeks ago    161MB
```

◆ Part 2: Custom Nginx Using Ubuntu Base Image

Step 1: Create Dockerfile

Create a Dockerfile using Ubuntu as the base image and install Nginx in it.



Step 2: Build Custom Image

Build a Docker image using the created Dockerfile.

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp3file> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ ls
Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ docker build -t nginx-ubuntu
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:22.04
----> 65c77cbc27c2
Step 2/4 : RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lists/*
----> Running in 3bc37fe0dd75
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1297 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3677 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [62.6 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6388 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 Packages [1792 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4014 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1608 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [6607 kB]
```

Step 3: Run the Container

Start a container using the custom Ubuntu-based image.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu  
bf4600f5806d78069e3df48880fbba2c46ef86ff13dc1c04903c210f8
```

Step 4: Verify Output

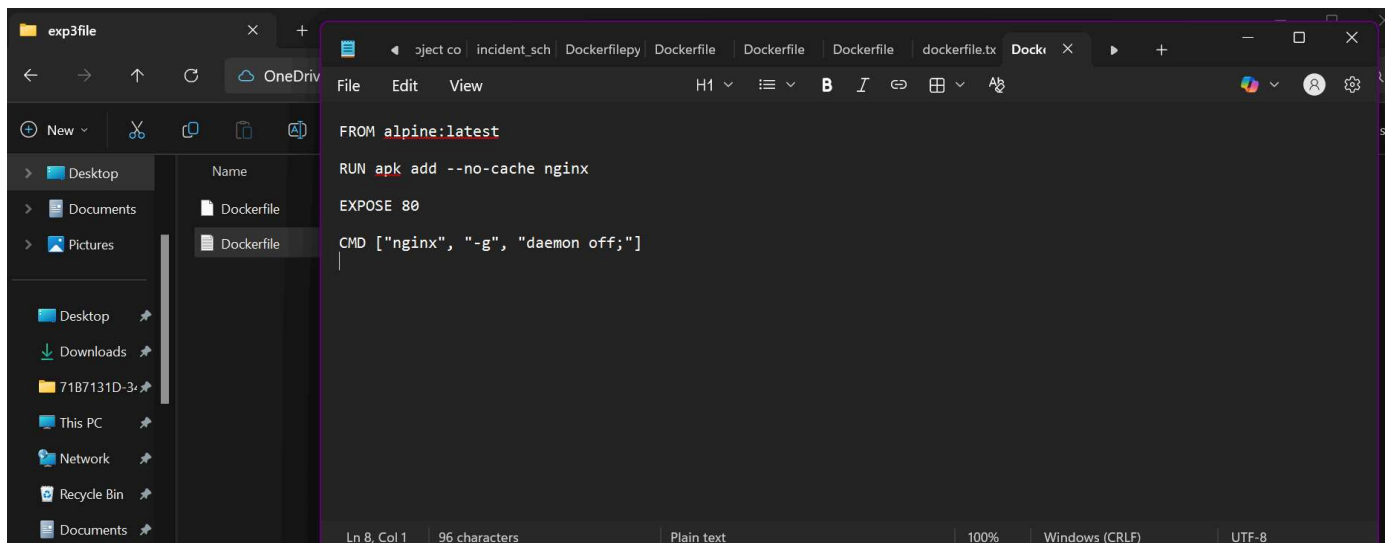
Check the Nginx webpage in the browser.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ docker images nginx-ubuntu  
REPOSITORY TAG IMAGE ID CREATED SIZE  
nginx-ubuntu latest 56270f5b5384 32 seconds ago 134MB
```

◆ Part 3: Custom Nginx Using Alpine Base Image

Step 1: Create Dockerfile

Create a Dockerfile using Alpine Linux as the base image and install Nginx.



The screenshot shows a code editor with a Dockerfile. The file is named 'Dockerfile' and is located in the 'exp3file' directory. The content of the Dockerfile is as follows:

```
FROM alpine:latest  
RUN apk add --no-cache nginx  
EXPOSE 80  
CMD ["nginx", "-g", "daemon off;"]
```

The editor interface includes a file explorer on the left showing the 'exp3file' directory and its contents. The status bar at the bottom indicates the file is at line 8, column 1, with 96 characters, in plain text format, with a 100% zoom level, using Windows (CRLF) line endings, and UTF-8 encoding.

Step 2: Build Image

Build the Docker image using the Alpine-based Dockerfile.

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp3file> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ ls
Dockerfile Dockerfile.txt
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ mv Dockerfile.txt Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ ls
Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ docker build -t nginx-alpine .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 2.048kB
Step 1/4 : FROM alpine:latest
latest: Pulling from library/alpine
589002ba0eae: Pulling fs layer
589002ba0eae: Verifying Checksum
589002ba0eae: Download complete
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
----> a40c03cbb81c
Step 2/4 : RUN apk add --no-cache nginx
----> Running in 01dc7b160c8a
(1/2) Installing pcre2 (10.47-r0)
(2/2) Installing nginx (1.28.2-r0)
      Executing nginx-1.28.2-r0.pre-install
      Executing nginx-1.28.2-r0.post-install
      Executing busybox-1.37.0-r30.trigger
```

Step 3: Run Container

Run a container using the newly created Alpine-based image.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ docker run -d --name nginx-alpine -p 808
2:80 nginx-alpine
b1bc0dec7827df00e4ecd14e5fadaa8542306298fb9f6503d46f557cd37e65c
```

Step 4: Verify Deployment

Open the browser and verify the Nginx welcome page.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ docker images nginx-alpine
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx-alpine latest 166a82683767 56 seconds ago 10.4MB
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp3file$ |
```



Result

Nginx was successfully deployed using:

- Official Docker Image
- Ubuntu Base Image
- Alpine Base Image

All containers were verified through browser testing.



Conclusion

This experiment demonstrated the deployment of Nginx using different Docker images and base operating systems.

From the comparison, it is observed that:

- The **Ubuntu-based image** has the largest size due to more built-in packages.
- The **Official Nginx image** provides a balance between size and usability.
- The **Alpine-based image** is the smallest and most efficient.

Therefore, Alpine-based images are preferred for lightweight and production environments, while Ubuntu-based images are useful for development and debugging purposes.

containerization-and-devops



Experiment 4: Docker Essentials

PART A: Containerizing Applications with Dockerfile

Step 1: Create Simple Flask App

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4> mkdir my-flask-app
```

```
Directory: C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4
```

Mode	LastWriteTime	Length	Name
d----	21-02-2026 23:11		my-flask-app

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4> cd my-flask-app
```

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4\my-flask-app> |
```

 app.py - C:/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/app.py (3.13.2)

File Edit Format Run Options Window Help

```
from flask import Flask
app = Flask(__name__)

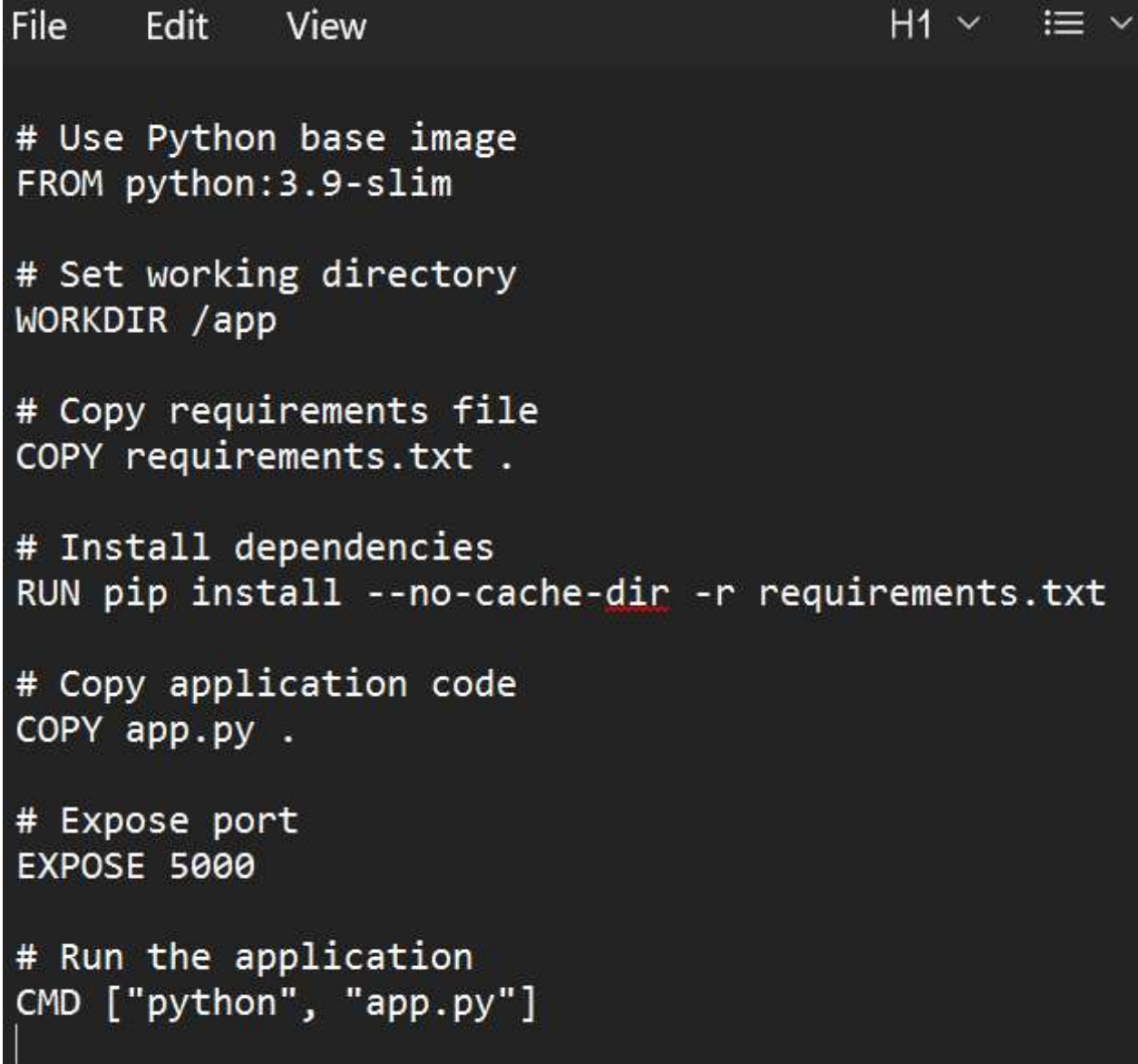
@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

This screenshot shows the creation of a simple Flask application.

Step 2: Create Dockerfile

A screenshot of a code editor with a dark theme. The editor has a menu bar at the top with 'File', 'Edit', and 'View'. On the right side of the menu bar, there is a zoom level 'H1' with a dropdown arrow, and a hamburger menu icon with a dropdown arrow. The main area of the editor contains a Dockerfile with the following content:

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

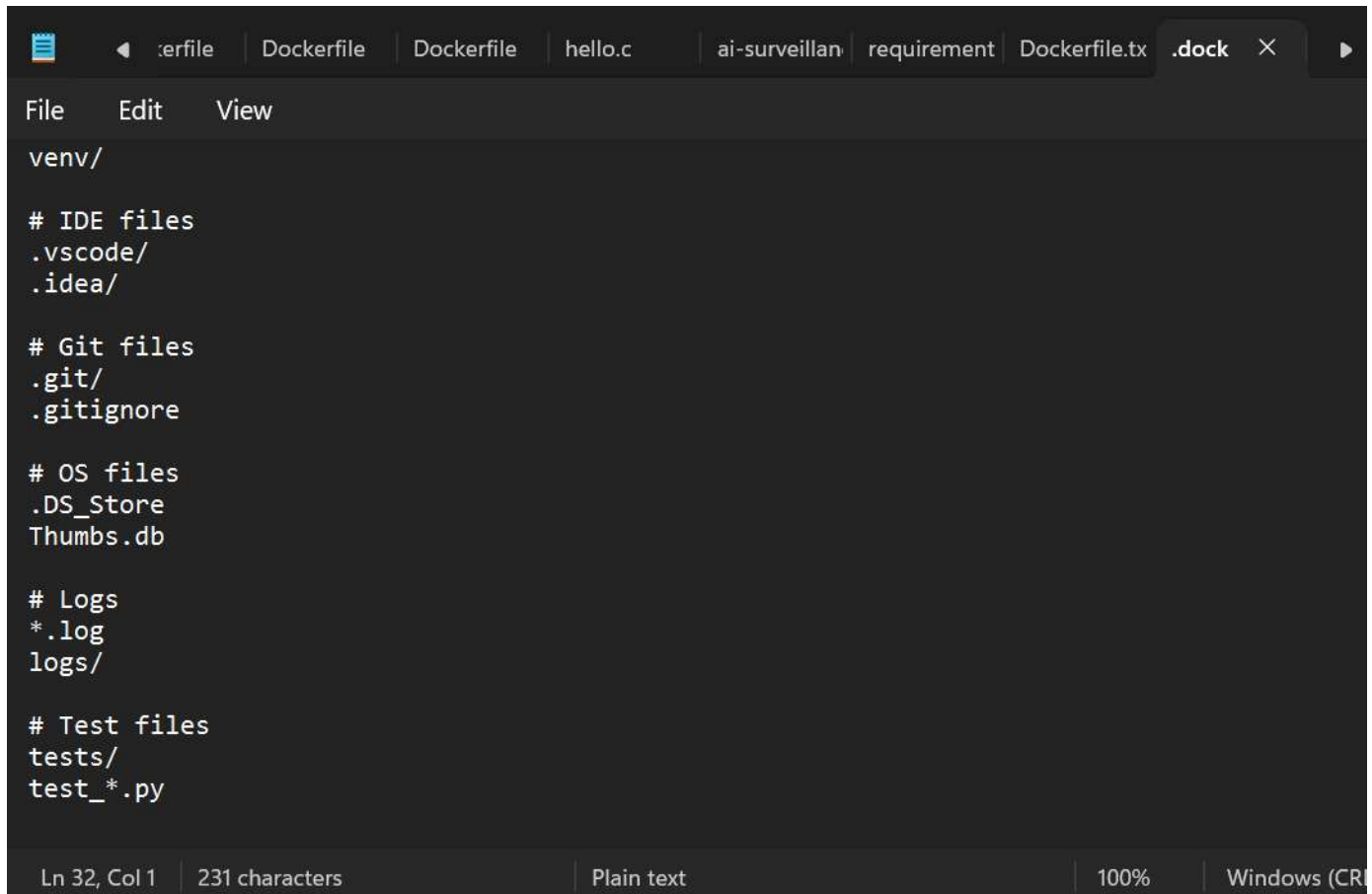
# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
|
```

This Dockerfile is used to containerize the Flask application.

PART 2: Using .dockerignore



```

venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

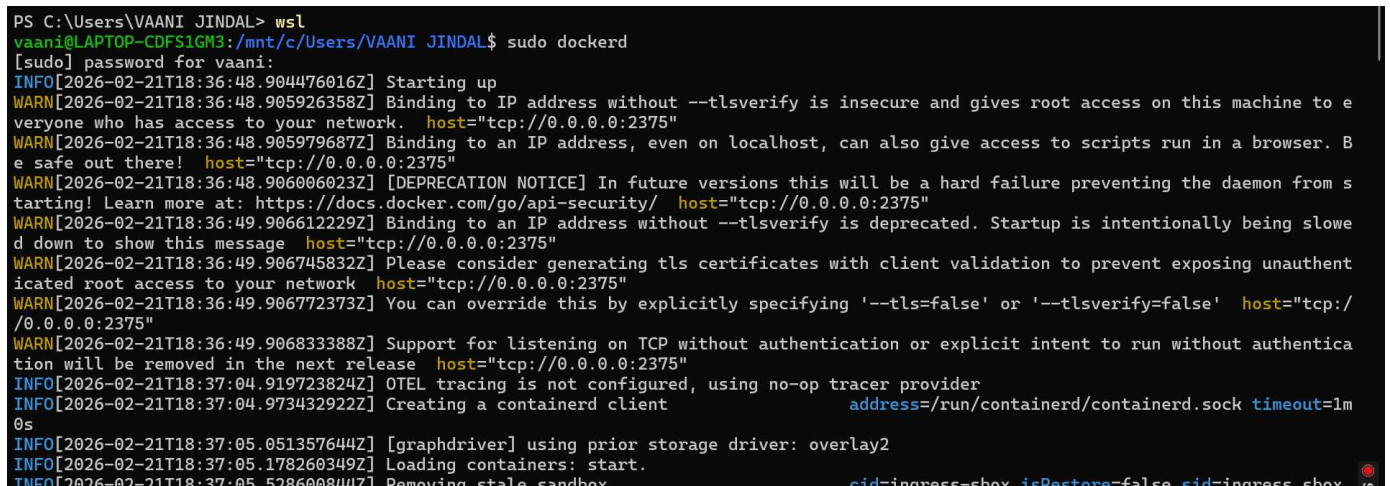
# Test files
tests/
test_*.py

```

Ln 32, Col 1 | 231 characters | Plain text | 100% | Windows (CR)

The `.dockerignore` file is used to exclude unnecessary files from the Docker build context.

PART 3:



```

PS C:\Users\VAANI JINDAL> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo dockerd
[sudo] password for vaani:
INFO[2026-02-21T18:36:48.904476016Z] Starting up
WARN[2026-02-21T18:36:48.905926358Z] Binding to IP address without --tlsverify is insecure and gives root access on this machine to e
everyone who has access to your network. host="tcp://0.0.0.0:2375"
WARN[2026-02-21T18:36:48.905979687Z] Binding to an IP address, even on localhost, can also give access to scripts run in a browser. B
e safe out there! host="tcp://0.0.0.0:2375"
WARN[2026-02-21T18:36:48.906006023Z] [DEPRECATION NOTICE] In future versions this will be a hard failure preventing the daemon from s
tarting! Learn more at: https://docs.docker.com/go/api-security/ host="tcp://0.0.0.0:2375"
WARN[2026-02-21T18:36:49.906612229Z] Binding to an IP address without --tlsverify is deprecated. Startup is intentionally being slowe
d down to show this message host="tcp://0.0.0.0:2375"
WARN[2026-02-21T18:36:49.906745832Z] Please consider generating tls certificates with client validation to prevent exposing unauthent
icated root access to your network host="tcp://0.0.0.0:2375"
WARN[2026-02-21T18:36:49.906772373Z] You can override this by explicitly specifying '--tls=false' or '--tlsverify=false' host="tcp://
0.0.0.0:2375"
WARN[2026-02-21T18:36:49.906833388Z] Support for listening on TCP without authentication or explicit intent to run without authentica
tion will be removed in the next release host="tcp://0.0.0.0:2375"
INFO[2026-02-21T18:37:04.919723824Z] OTEL tracing is not configured, using no-op tracer provider
INFO[2026-02-21T18:37:04.973432922Z] Creating a containerd client address=/run/containerd/containerd.sock timeout=1m
0s
INFO[2026-02-21T18:37:05.051357644Z] [graphdriver] using prior storage driver: overlay2
INFO[2026-02-21T18:37:05.178260349Z] Loading containers: start.
INFO[2026-02-21T18:37:05.528600844Z] Removing stale sandbox

```

Step 1: Basic Build Command

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ cd "/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app"
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -t my-flask-app .
Dockerfile.txt app.py requirements.txt.txt
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ mv Dockerfile.txt Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ mv requirements.txt.txt requirements.txt
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ ls
Dockerfile app.py requirements.txt
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -t my-flask-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 5.12kB
Step 1/7 : FROM python:3.9-slim
3.9-slim: Pulling from library/python

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	latest	11130f8b2887	About a minute ago	132MB
c-app	1.0	310db60249ee	11 days ago	900kB
<none>	<none>	0014a9b0e99b	11 days ago	334MB
nginx-alpine	latest	166a82683767	2 weeks ago	10.4MB
nginx-ubuntu	latest	56270f5b5384	2 weeks ago	134MB
python-app	1.0	e8f0986db1f1	2 weeks ago	1.21GB
mysql	latest	f66b7a288113	2 weeks ago	922MB
nginx	latest	5cdef4ac3335	2 weeks ago	161MB
python	latest	9a7c5808942d	2 weeks ago	1.12GB
nginx	<none>	248d2326f351	2 weeks ago	161MB
Vaanijindal22/javaapp	2.0	1a51016cea2d	3 weeks ago	773MB
javaapp	2.0	1a51016cea2d	3 weeks ago	773MB
vaanijindal22/javaapp	2.0	1a51016cea2d	3 weeks ago	773MB
Vaanijindal22/java-app	1.0	dd28af5caa74	3 weeks ago	773MB
java-app	1.0	dd28af5caa74	3 weeks ago	773MB
alpine	latest	a40c03cbb81c	3 weeks ago	8.44MB
myrepo/java-app	1.0	827e87483043	3 weeks ago	922MB
ubuntu	latest	493218ed0f40	5 weeks ago	78.1MB
nginx	<none>	4af177a024eb	5 weeks ago	161MB
ubuntu	22.04	65c77cbc27c2	6 weeks ago	77.9MB
python	3.9-slim	085da638e1b8	3 months ago	122MB
hello-world	latest	1b44b5a3e06a	6 months ago	10.1kB
busybox	latest	af3f0f48a24e	17 months ago	4.43MB

This shows the `docker build` command used to create a Docker image.

Step 2: Tagging Image

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -t my-flask-app:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> e6fc41a8a827
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 66e860ebf3d2
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 1ab9592c1530
Step 5/7 : COPY app.py .
--> Using cache
--> da3198b5f5f4
Step 6/7 : EXPOSE 5000
--> Using cache
--> 9ad699310b43
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> 11130f8b2887
Successfully built 11130f8b2887
Successfully tagged my-flask-app:1.0

```

This shows how Docker images are tagged.

Multiple Tags:

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -t my-flask-app:latest
test -t my-flask-app:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  5.12kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> e6fc41a8a827
Step 3/7 : COPY requirements.txt .
----> Using cache
----> 66e860ebf3d2
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 1ab9592c1530
Step 5/7 : COPY app.py .
----> Using cache
----> da3198b5f5f4
Step 6/7 : EXPOSE 5000
----> Using cache
----> 9ad699310b43
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 11130f8b2887
Successfully built 11130f8b2887
Successfully tagged my-flask-app:latest
Successfully tagged my-flask-app:1.0
```

Assigning multiple tags to the same image.

Tag with Custom Registry:

```
Successfully tagged my-flask-app:1.0
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -t username/my-flask-app:1.0
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  5.12kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> e6fc41a8a827
Step 3/7 : COPY requirements.txt .
----> Using cache
----> 66e860ebf3d2
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 1ab9592c1530
Step 5/7 : COPY app.py .
----> Using cache
----> da3198b5f5f4
Step 6/7 : EXPOSE 5000
----> Using cache
----> 9ad699310b43
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> 11130f8b2887
Successfully built 11130f8b2887
Successfully tagged username/my-flask-app:1.0
```

Tagging image with a custom registry name.

Tag Existing Image:

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker tag my-flask-app:latest my-flask-app:v1.0
```

Tagging an already built Docker image.

Step 3: View Image Details

Show Image History

```

vaani@LAPTOP-CDFS1GM3: /mnt/c/Users/VAANI_JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker history my-flask-app
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
11130f8b2887   4 minutes ago   /bin/sh -c #(nop)  CMD ["python" "app.py"]   0B        buildkit.dockerfile.v0
9ad699310b43   4 minutes ago   /bin/sh -c #(nop)  EXPOSE 5000              0B        buildkit.dockerfile.v0
da3198b5f5f4   4 minutes ago   /bin/sh -c #(nop)  COPY file:49edba258cd3ee6c... 241B      buildkit.dockerfile.v0
1ab9592c1530   4 minutes ago   /bin/sh -c pip install --no-cache-dir -r req... 10.5MB    buildkit.dockerfile.v0
66e860ebf3d2   4 minutes ago   /bin/sh -c #(nop)  COPY file:184e5856826c31af... 14B       buildkit.dockerfile.v0
e6fc41a8a827   4 minutes ago   /bin/sh -c #(nop)  WORKDIR /app             0B        buildkit.dockerfile.v0
085da638e1b8   3 months ago    CMD ["python3"]         0B        buildkit.dockerfile.v0
<missing>      3 months ago    RUN /bin/sh -c set -eux; for src in idle3 p... 36B       buildkit.dockerfile.v0
<missing>      3 months ago    RUN /bin/sh -c set -eux; savedAptMark="$(a... 39.5MB    buildkit.dockerfile.v0
<missing>      3 months ago    ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8... 0B        buildkit.dockerfile.v0
<missing>      3 months ago    ENV PYTHON_VERSION=3.9.25 0B        buildkit.dockerfile.v0
<missing>      3 months ago    ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3... 0B        buildkit.dockerfile.v0
<missing>      3 months ago    RUN /bin/sh -c set -eux; apt-get update; a... 3.81MB    buildkit.dockerfile.v0
<missing>      3 months ago    ENV LANG=C.UTF-8        0B        buildkit.dockerfile.v0
<missing>      3 months ago    ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... 0B        buildkit.dockerfile.v0
<missing>      4 months ago    # debian.sh --arch 'amd64' out/ 'trixie' '@1... 78.6MB    debuerreotype 0.16

```

Displays the image layer history.

Inspect Image Details

```

vaani@LAPTOP-CDFS1GM3: /mnt/c/Users/VAANI_JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker inspect my-flask-app
[
  {
    "Id": "sha256:11130f8b28877a9b5326e250df817fb4fdac6295f6f118f965b03a7a0f52edb6",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0",
      "username/my-flask-app:1.0"
    ],
    "RepoDigests": [],
    "Parent": "sha256:9ad699310b438e32924a6de51ab274fdd6e39f5405588c6fb632f8740a306b4e",
    "Comment": "",
    "Created": "2026-02-21T18:44:39.388524276Z",
    "DockerVersion": "28.2.2",
    "Author": "",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "Cmd": [
        "python",
        "app.py"
      ],
      "WorkingDir": "/app"
    }
  }
]

```

Shows detailed metadata of the Docker image.

PART 4: Running Containers

Step 1: Run Container

```

PS C:\Users\VAANI JINDAL> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ cd "/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app"
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker run -d -p 5000:5000 --name flask-container my-flask-app
3f692469c1241bf60d502eafe0174734dd6676ca116845e824f0ec521d4ec75c
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ curl http://localhost:5000
Hello from Docker!vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
3f692469c124   my-flask-app   "python app.py"         29 seconds ago Up 29 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [22/Feb/2026 14:35:15] "GET / HTTP/1.1" 200 -

```

This screenshot shows how to run a Docker container.

Step 2: Manage Container

```

172.17.0.1 - - [22/Feb/2026 14:35:15] "GET / HTTP/1.1" 200 -
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker stop flask-container
flask-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker start flask-container
flask-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker rm flask-container
Error response from daemon: cannot remove container "flask-container": container is running: stop the container before removing or force remove
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker rm -f flask-container
flask-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$

```

Commands used to manage running containers.

PART 5: Multistage Builds

Step: Simple Multistage Dockerfile

```
File    Edit    View

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

This shows a multistage Dockerfile.

Step 3: Build and Compare

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -t flask-regular
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> e6fc41a8a827
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 66e860ebf3d2
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 1ab9592c1530
Step 5/7 : COPY app.py .
--> Using cache
--> da3198b5f5f4
Step 6/7 : EXPOSE 5000
--> Using cache
--> 9ad699310b43
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> 11130f8b2887
Successfully built 11130f8b2887
Successfully tagged flask-regular:latest
```


Building multistage Docker image.

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI_JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 6.656kB
Step 1/15 : FROM python:3.9-slim AS builder
----> 085da638e1b8
Step 2/15 : WORKDIR /app
----> Using cache
----> e6fc41a8a827
Step 3/15 : COPY requirements.txt .
----> Using cache
----> 66e860ebf3d2
Step 4/15 : RUN python -m venv /opt/venv
----> Running in 87cf36ed207f
----> Removed intermediate container 87cf36ed207f
----> 5caec31cae1f
Step 5/15 : ENV PATH="/opt/venv/bin:$PATH"
----> Running in afe596a967e2
----> Removed intermediate container afe596a967e2
----> 5f843fb67938
Step 6/15 : RUN pip install --no-cache-dir -r requirements.txt
----> Running in 699c37fb7a6c
Collecting Flask==2.3.3
  Downloading flask-2.3.3-py3-none-any.whl (96 kB)
    96.1/96.1 kB 850.3 kB/s eta 0:00:00
Collecting itsdangerous>=2.1.2
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting importlib-metadata>=3.6.0
  Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
Successfully built 0cad66b02dba
Successfully tagged flask-multistage:latest
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI_JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker images | grep flask-
flask-multistage      latest      0cad66b02dba   26 seconds ago   145MB
flask-regular         latest      11130f8b2887   20 hours ago     132MB
my-flask-app          1.0        11130f8b2887   20 hours ago     132MB
my-flask-app          latest     11130f8b2887   20 hours ago     132MB
my-flask-app          v1.0       11130f8b2887   20 hours ago     132MB
username/my-flask-app 1.0        11130f8b2887   20 hours ago     132MB

```

Comparison between normal and multistage image sizes.

PART 6: Publishing to Docker Hub

Step 1: Prepare for Publishing


```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker login
Authenticating with existing credentials... [Username: vaanijindal22]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker tag my-flask-app:latest
vaanijindal22/my-flask-app:1.0
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker tag my-flask-app:latest
vaanijindal22/my-flask-app:latest
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker push vaanijindal22/my-fl
ask-app:1.0
The push refers to repository [docker.io/vaanijindal22/my-flask-app]
347aa73cd309: Pushed
f61d93a3018c: Pushed
40a2f9dd5604: Pushed
c4d7a0626679: Pushed
c8f6b54339a8: Pushed
298992e09a03: Pushed
4f237755fbae: Pushed
d7c97cb6f1fe: Pushed
1.0: digest: sha256:491e57c1bf2970dc96090fa475005e6dbf582fc89410b2c13f9d946c84a74844 size: 1990
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker push vaanijindal22/my-fl
ask-app:latest
The push refers to repository [docker.io/vaanijindal22/my-flask-app]
347aa73cd309: Layer already exists
f61d93a3018c: Layer already exists
40a2f9dd5604: Layer already exists
c4d7a0626679: Layer already exists
c8f6b54339a8: Layer already exists
298992e09a03: Layer already exists
4f237755fbae: Layer already exists
d7c97cb6f1fe: Layer already exists
latest: digest: sha256:491e57c1bf2970dc96090fa475005e6dbf582fc89410b2c13f9d946c84a74844 size: 1990

```

Login to Docker Hub before pushing images.

Step 2: Pull and Run from Docker Hub

```

latest: digest: sha256:491e57c1bf2970dc96090fa475005e6dbf582fc89410b2c13f9d946c84a74844 size: 1990
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker pull vaanijindal22/my-fl
ask-app:latest
latest: Pulling from vaanijindal22/my-flask-app
Digest: sha256:491e57c1bf2970dc96090fa475005e6dbf582fc89410b2c13f9d946c84a74844
Status: Image is up to date for vaanijindal22/my-flask-app:latest
docker.io/vaanijindal22/my-flask-app:latest
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ docker run -d -p 5000:5000 vaan
ijindal22/my-flask-app:latest
71ad083e4a7580b528190895e7521aca3a3671119d3c55947712e40b1640947b
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-flask-app$ |

```

PART 7: Node.js Example

Step 1: Node.js Application

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4> mkdir my-node-app

Directory: C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4


Mode                LastWriteTime         Length Name
----                -
d-----          22-02-2026         20:26         my-node-app

PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4> cd my-node-app
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp4\my-node-app> |
```

Shows Node.js project structure.

##step 2: Create app.js, package.json, Node.js Dockerfile

Name	Status	Date modified	Type	Size
app.js	✓	22-02-2026 20:28	JSFile	1 KB
Dockerfile	✓	22-02-2026 20:31	File	1 KB
package	✓	22-02-2026 20:29	JSON Source File	1 KB

Displays Node.js application files.

step 3: build and run

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ mv "Node.js Dockerfile.txt" Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker build -t my-node-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
25ff2da83641: Download complete
f18232174bc9: Verifying Checksum
f18232174bc9: Download complete
f18232174bc9: Pull complete
dd71dde834b5: Verifying Checksum
dd71dde834b5: Download complete
dd71dde834b5: Pull complete
1e5a4c89cee5: Pull complete
25ff2da83641: Pull complete
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
--> ee77c6cd7c18
Step 2/7 : WORKDIR /app
--> Running in bcecec91f3ec

Successfully tagged my-node-app:latest
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker run -d -p 3000:3000 --name node-container my-node-app
df3926cbf68d97020c7431c9ef7f3edf74e18593a2ad86259590fa44137fb721
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ curl http://localhost:3000
Hello from Node.js Docker!vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ |
```

Successful execution of Node.js container.

containerization-and-devops



Experiment 5: Docker Volumes, Environment Variables, Monitoring & Networks

Reference Link:

<https://upessocs.github.io/#dir=/Lectures/Containerization%20and%20DevOps/Lab/&file=Experiment%202005%20Docker%20Volumes,%20Environment%20Variables,%20Monitoring,%20Networks%20Final.md>

PART 1: Understanding Volume Persistence

Lab 1: Volume Persistence

```
PS C:\Users\VAANI JINDAL> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -it --name test-container ubuntu /bin/bash
root@e90786683176:/# echo "Hello World" > /data/message.txt
bash: /data/message.txt: No such file or directory
root@e90786683176:/# mkdir /data
root@e90786683176:/# echo "Hello World" > /data/message.txt
root@e90786683176:/# cat /data/message.txt
Hello World
root@e90786683176:/# docker start test-container
bash: docker: command not found
root@e90786683176:/# exit
exit
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker start test-container
test-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker exec test-container cat /data/message.txt
Hello World
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm test-container
Error response from daemon: cannot remove container "test-container": container is running: stop the container before removing or force remove
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo docker rm test-container
[sudo] password for vaani:
Error response from daemon: cannot remove container "test-container": container is running: stop the container before removing or force remove
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker stop test-container
test-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm test-container
test-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS      PORTS
jovial_kare
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -it --name test-container ubuntu /bin/bash
root@eb5b3f43c686:/# cat /data/message.txt
cat: /data/message.txt: No such file or directory
root@eb5b3f43c686:/#
```

This screenshot demonstrates Docker volume persistence.

Lab 2: Volume Types

a) Anonymous Volumes

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d -v /app/data --name web1 nginx
1baa110c9129435dc8160381fa785a9b97b5e247a45ac80d4deb111a734960eb
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume ls
DRIVER      VOLUME NAME
local       0d86627c85ce8c4e5c891bf0a5f8710835ed9366ea8b8aacf81537eb70b886b4
local       0e6c339a392d4bc0707750699b2a6a1e6763bfa3ac7c4d06b9f21662c20b1c79
local       3b15be5fbb08e2cb8abb0d45b744c44de1e8eb2be52fe4e9cdc46877c06b7489
local       16b9d11543d382153f5059b346db3eda988b294661e47d2ab69a1daf0ca75bc4
local       26febclasstask_mysql_data
local       26febclasstask_wp_content
local       55b372250c1a88ff54acc91c56eefc6111ed1224f5073a69943d76414a2cd70a
local       80f158dea58688cfd7a2cec4edb6130e7f332aa5132570ad0fd3c1ae4739f6c7
local       573e0f8b12cbc2a0eb08dfccea233c022dd633f66de7f056780aa173b6cc547b
local       587a13f0868637dc82ab0448bd380bd469e395c1d8d22f45d37c0e1e848146f1
local       a39c6a7fdec1bcc893e84ec7bbe8e0363b60849376531ef79896d6747478e02f
local       c830fdbb972d39bb08d81d6f15ad4362db939e1452e9d7caeb0882f679d228ec8
local       classtask25feb_mysql_data
local       classtask25feb_wp_content
local       da80498974262dd88c092458cf498b778d25ca89f38496e5da10acc46b899587
local       docker-postgres-project_postgres_data
local       e66c8a4683852864b24f0ff9d20b14a2083233d2a84a4cea2baafe3db15de984
local       e43687703f2c4e79f8a5771d9c3f39096c09a814476347b442b177fce95bcfc8
local       k3d-mycluster-images
local       myvolume
local       portainer_data
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect web1 | grep -A 5 Mounts
  "Mounts": [
    {
      "Type": "volume",
      "Name": "e66c8a4683852864b24f0ff9d20b14a2083233d2a84a4cea2baafe3db15de984",
      "Source": "/var/lib/docker/volumes/e66c8a4683852864b24f0ff9d20b14a2083233d2a84a4cea2baafe3db15de984/_data",
      "Destination": "/app/data",

```

Anonymous volumes are automatically created by Docker.

b) Named Volumes


```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume create mydata
mydata
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d -v mydata:/app/data --name web2 nginx
91bb6ab46a361f76109dd6ce48ca92d025058729c2a2cb9c12bd709db1ef5e90
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume ls
DRIVER      VOLUME NAME
local       0d86627c85ce8c4e5c891bf0a5f8710835ed9366ea8b8aacf81537eb70b886b4
local       0e6c339a392d4bc0707750699b2a6a1e6763bfa3ac7c4d06b9f21662c20b1c79
local       3b15be5fbb08e2cb8abb0d45b744c44de1e8eb2be52fe4e9cdc46877c06b7489
local       16b9d11543d382153f5059b346db3eda988b294661e47d2ab69a1daf0ca75bc4
local       26febclasstask_mysql_data
local       26febclasstask_wp_content
local       55b372250c1a88ff54acc91c56eefc6111ed1224f5073a69943d76414a2cd70a
local       80f158dea58688cfd7a2cec4edb6130e7f332aa5132570ad0fd3c1ae4739f6c7
local       573e0f8b12cbc2a0eb08dfccea233c022dd633f66de7f056780aa173b6cc547b
local       587a13f0868637dc82ab0448bd380bd469e395c1d8d22f45d37c0e1e848146f1
local       a39c6a7fdec1bcc893e84ec7bbe8e0363b60849376531ef79896d6747478e02f
local       c830fdbb972d39bb08d81d6f15ad4362db939e1452e9d7caeb0882f679d228ec8
local       classtask25feb_mysql_data
local       classtask25feb_wp_content
local       da80498974262dd88c092458cf498b778d25ca89f38496e5da10acc46b899587
local       docker-postgres-project_postgres_data
local       e66c8a4683852864b24f0ff9d20b14a2083233d2a84a4cea2baafe3db15de984
local       e43687703f2c4e79f8a5771d9c3f39096c09a814476347b442b177fce95bcfc8
local       k3d-mycluster-images
local       mydata
local       myvolume
local       portainer_data
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume inspect mydata
[
  {
    "CreatedAt": "2026-03-27T08:56:33Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
  }
]

```

Named volumes allow reuse and persistence.

c) Bind Mounts

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ mkdir ~/myapp-data
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d -v ~/myapp-data:/app/data --name web3 nginx
e193e6b7f9a5129e21a8c363586f3a48caf7d8c9b85bc25e10c737c78df30bd7
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ echo "From Host" > ~/myapp-data/host-file.txt
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker exec web3 cat /app/data/host-file.txt
From Host
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |

```

Bind mounts map host directories.

Lab 3: Practical Examples

Ex 1: Database with Persistent Storage


```
From Host
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
d4c7048d1cf1: Pull complete
6375ec02106f: Pull complete
771773a1fe7a: Pull complete
c3b8cfb10b41: Pull complete
1378718ee973: Pull complete
5915b0e5580c: Pull complete
60c3210ab1f6: Pull complete
440208b3f29f: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd000
Status: Downloaded newer image for mysql:8.0
9e5c298b3624591e3ec981e81e08c78ff4f5b2d1ba792c7d51fc2e4470762822
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker stop mysql-db
mysql-db
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm mysql-db
mysql-db
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
```

Database container with persistent volume.

Ex 2: Web App with Configuration Files

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ echo 'server {
listen 80;
server_name localhost;
location / {
    return 200 "Hello from mounted config!";
}
}' > ~/nginx-config/nginx.conf
-bash: /home/vaani/nginx-config/nginx.conf: Is a directory
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ cat ~/nginx-config/nginx.conf
cat: /home/vaani/nginx-config/nginx.conf: Is a directory
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ls -l ~/nginx-config
total 4
drwxr-xr-x 2 root root 4096 Mar 27 09:10 nginx.conf
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ rm -r ~/nginx-config/nginx.conf
rm: remove write-protected directory '/home/vaani/nginx-config/nginx.conf'? yes
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ echo 'server {
listen 80;
server_name localhost;
location / {

```

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker start nginx-custom
nginx-custom
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
c2ecdeeb400d   nginx         "/docker-entrypoint..." 5 minutes ago  Up 9 seconds  0.0.0.0:8080->80/tcp, [::]:
a422968c84a2   mysql:8.0     "docker-entrypoint.s..." 7 minutes ago  Up 7 minutes  3306/tcp, 33060/tcp
e193e6b7f9a5   nginx         "/docker-entrypoint..." 18 minutes ago  Up 18 minutes  80/tcp
01bb6ab46a36   nginx         "/docker-entrypoint..." 19 minutes ago  Up 19 minutes  80/tcp
1baa110c9129   nginx         "/docker-entrypoint..." 21 minutes ago  Up 21 minutes  80/tcp
2ecff23c2c7d   portainer/portainer-ce:lts "/portainer"           7 days ago    Up 32 minutes  0.0.0.0:8000->8000/tcp, [::]:
f1c8e59c370b   ghcr.io/k3d-io/k3d-proxy:5.8.3 "/bin/sh -c nginx-pr..." 9 days ago    Up 32 minutes  80/tcp, 0.0.0.0:43783->6443
/tcp
fc23f654b650   rancher/k3s:v1.31.5-k3s1  "/bin/k3d-entrypoint..." 9 days ago    Up 32 minutes  k3d-mycluster-serverlb
k3d-mycluster-server-0
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ curl localhost:8080
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |

```

Web app using bind mounts.

Lab 4: Volume Management Commands

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume create app-volume
app-volume
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-03-27T09:20:41Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
3b15be5fbb08e2cb8abb0d45b744c44de1e8eb2be52fe4e9cdc46877c06b7489
e43687703f2c4e79f8a5771d9c3f39096c09a814476347b442b177fce95bcfc8

Total reclaimed space: 123MB
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker volume rm volume-name
Error response from daemon: volume volume-name not found
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker cp local-file.txt container-name:/path/in/volume
lsat /mnt/c/Users/VAANI JINDAL/local-file.txt: no such file or directory
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ^C
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ pwd
/mnt/c/Users/VAANI JINDAL
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ls

```

```

aani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ls
AppData
Application Data'
Contacts
Cookies
Documents
Downloads
Favorites
IntelGraphicsProfiles
Links
Local Settings'
My Documents'
NTUSER.DAT
NTUSER.DAT{e87d006b-115d-11f1-a5ee-00155d6a868b}.TM.blf
NTUSER.DAT{e87d006b-115d-11f1-a5ee-00155d6a868b}.TMContainer000000000000000001.regtrans-ms
NTUSER.DAT{e87d006b-115d-11f1-a5ee-00155d6a868b}.TMContainer000000000000000002.regtrans-ms
NetHood
OneDrive
PrintHood
Recent
SendTo
'Start Menu'
Templates
VirtualBox VMs
ai-smart-surveillance
container.tar
docker-postgres-project
html
image.tar
java-app.tar
myvolume
ntuser.dat.LOG1
ntuser.dat.LOG2
ntuser.ini
streamlit_app
aani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ echo "Hello from Docker Volume" > local-file.txt
aani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ cat local-file.txt
ello from Docker Volume

```

Volume management commands.

PART 2: Environment Variables

Method 1: Using -e flag

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
--name app3 \
-e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
-e DEBUG="true" \
-p 3000:3000 \
my-node-app
745013441b2ac4cb696ea55f8447582b51270d66256a05ddc90c9835c2ba446c
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
-e VAR1=value1 \
-e VAR2=value2 \
-e VAR3=value3 \
my-app4
Unable to find image 'my-app4:latest' locally
docker: Error response from daemon: pull access denied for my-app4, repository does not exist or may require 'docker login': denied:
requested access to the resource is denied

Run 'docker run --help' for more information
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
-e VAR1=value1 \
-e VAR2=value2 \
-e VAR3=value3 \
my-node-app
43b7cbe72e9e497b0e84fb7422810b11ea270bc8e8026d9127c579751aee6d2
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$

```

Using environment variables with -e flag.

Method 2: Using env file

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp 5 files$ docker build -t my-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.608kB
Step 1/11 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/11 : ENV PYTHONUNBUFFERED=1
--> Using cache
--> 344b59eb9ca9
Step 3/11 : ENV PYTHONDONTWRITEBYTECODE=1
--> Using cache
--> ed4e0a266e96
Step 4/11 : WORKDIR /app
--> Using cache
--> cf5c5a199586
Step 5/11 : COPY requirements.txt .

```

Using environment variable file.

PART 3: Docker Monitoring

Lab 1: Basic Monitoring Commands

Docker Stats

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
b43b7cbe72e9	funny_hodgkin	0.00%	15.59MiB / 7.641GiB	0.20%	866B / 126B	0B / 0B	7
f45013441b2a	app3	0.00%	19.02MiB / 7.641GiB	0.24%	992B / 126B	49.4MB / 0B	7
c2ecdeeb400d	nginx-custom	0.00%	8.449MiB / 7.641GiB	0.11%	1.96kB / 670B	1.62MB / 8.19kB	9
a422968c84a2	new-mysql	0.90%	349.5MiB / 7.641GiB	4.47%	1.57kB / 126B	65.5kB / 15.1MB	37
e193e6b7f9a5	web3	0.00%	7.688MiB / 7.641GiB	0.10%	1.82kB / 126B	49.2kB / 12.3kB	9
01bb6ab46a36	web2	0.00%	7.602MiB / 7.641GiB	0.10%	1.94kB / 126B	0B / 12.3kB	9
1baa110c9129	web1	0.00%	14.49MiB / 7.641GiB	0.19%	2.07kB / 126B	14.2MB / 12.3kB	9
2ecff23c2c7d	portainer	0.00%	14.88MiB / 7.641GiB	0.19%	3.17kB / 126B	65.9MB / 139kB	11
f1c8e59c370b	k3d-mycluster-serverlb	0.02%	25.14MiB / 7.641GiB	0.32%	1.64MB / 1.63MB	21.8MB / 16.4kB	19
fc23f654b650	k3d-mycluster-server-0	10.58%	787.3MiB / 7.641GiB	10.06%	1.75MB / 1.7MB	474MB / 118MB	559

Docker stats command.

Docker Stats Table

NAME	CPU %	MEM USAGE / LIMIT	NET I/O
funny_hodgkin	0.00%	15.59MiB / 7.641GiB	866B / 126B
app3	0.00%	19.04MiB / 7.641GiB	1.06kB / 126B
nginx-custom	0.00%	8.449MiB / 7.641GiB	1.96kB / 670B
new-mysql	0.92%	349.5MiB / 7.641GiB	1.57kB / 126B
web3	0.00%	7.688MiB / 7.641GiB	1.82kB / 126B
web2	0.00%	7.602MiB / 7.641GiB	1.94kB / 126B
web1	0.00%	14.49MiB / 7.641GiB	2.14kB / 126B
portainer	0.00%	14.88MiB / 7.641GiB	3.17kB / 126B
k3d-mycluster-serverlb	0.00%	25.14MiB / 7.641GiB	1.66MB / 1.65MB
k3d-mycluster-server-0	10.04%	788.4MiB / 7.641GiB	1.77MB / 1.72MB


```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker stats --no-stream
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O     PIDS
b43b7cbe72e9   funny_hodgkin       0.00%    15.61MiB / 7.641GiB  0.20%    866B / 126B   0B / 0B       7
f45013441b2a   app3                0.00%    19.04MiB / 7.641GiB  0.24%    1.06kB / 126B 49.4MB / 0B   7
c2ecdeeb400d   nginx-custom        0.00%    8.449MiB / 7.641GiB  0.11%    1.96kB / 670B 1.62MB / 8.19kB 9
a422968c84a2   new-mysql           1.08%    349.5MiB / 7.641GiB  4.47%    1.57kB / 126B 65.5kB / 15.1MB 37
e193e6b7f9a5   web3                0.00%    7.688MiB / 7.641GiB  0.10%    1.82kB / 126B 49.2kB / 12.3kB 9
01bb6ab46a36   web2                0.00%    7.602MiB / 7.641GiB  0.10%    1.94kB / 126B 0B / 12.3kB   9
1baa110c9129   web1                0.00%    14.49MiB / 7.641GiB  0.19%    2.14kB / 126B 14.2MB / 12.3kB 9
2ecff23c2c7d   portainer           0.02%    14.88MiB / 7.641GiB  0.19%    3.17kB / 126B 65.9MB / 139kB 11
f1c8e59c370b   k3d-mycluster-serverlb 0.00%    25.14MiB / 7.641GiB  0.32%    1.67MB / 1.66MB 21.8MB / 16.4kB 19
fc23f654b650   k3d-mycluster-server-0 10.11%    788.4MiB / 7.641GiB  10.08%    1.78MB / 1.73MB 474MB / 120MB  559

```

Docker stats output.

Lab 2: Process Monitoring

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker top container-name
Error response from daemon: No such container: container-name
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker top container2
Error response from daemon: container 3c2815e90ddf9360126f2e9dalb62ec046ef893c51dd6c8baa47186de7fa2a18 is not running
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker top container2 -ef
Error response from daemon: container 3c2815e90ddf9360126f2e9dalb62ec046ef893c51dd6c8baa47186de7fa2a18 is not running
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ps aux | grep docker
root      566   0.0  0.0  14316  7120 pts/0    S+   08:47   0:00 sudo dockerd
root      567   0.0  0.0  14316  2536 pts/2    Ss   08:47   0:00 sudo dockerd
root      568   1.0  0.3 5390476 104436 pts/2    Sl+  08:47   0:52 dockerd
root     1216   0.0  0.0   1056   640 ?        Ss   08:47   0:00 /sbin/docker-init -- /bin/sh -c nginx-proxy
root     1219   0.0  0.0   1056   640 ?        Ss   08:47   0:00 /sbin/docker-init -- /bin/k3d-entrypoint.sh server --tls-san 0.0.0.0
0.0 --tls-san k3d-mycluster-serverlb
root     1273   0.0  0.0  1745100 3968 pts/2    Sl+  08:47   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8000
-container-ip 172.17.0.2 -container-port 8000 -use-listen-fd
root     1281   0.0  0.0  1745100 4096 pts/2    Sl+  08:47   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8000 -cont
ainer-ip 172.17.0.2 -container-port 8000 -use-listen-fd
root     1291   0.0  0.0  1745100 3968 pts/2    Sl+  08:47   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9443
-container-ip 172.17.0.2 -container-port 9443 -use-listen-fd
root     1301   0.0  0.0  1745100 4224 pts/2    Sl+  08:47   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 9443 -cont
ainer-ip 172.17.0.2 -container-port 9443 -use-listen-fd
root     1350   0.0  0.0  1745100 4224 pts/2    Sl+  08:47   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 43783
-container-ip 172.25.0.3 -container-port 6443 -use-listen-fd
root     10016   0.0  0.0  1745100 4224 pts/2    Sl+  09:18   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080
-container-ip 172.17.0.7 -container-port 80 -use-listen-fd
root     10023   0.0  0.0  1745356 4096 pts/2    Sl+  09:18   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8080 -cont
ainer-ip 172.17.0.7 -container-port 80 -use-listen-fd
root     13298   0.0  0.0  1671368 3968 pts/2    Sl+  10:02   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 3000
-container-ip 172.17.0.8 -container-port 3000 -use-listen-fd
root     13305   0.0  0.0  1745100 3968 pts/2    Sl+  10:02   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 3000 -cont
ainer-ip 172.17.0.8 -container-port 3000 -use-listen-fd
vaani    13681   0.8  0.3 2144072 30712 pts/3    Sl+  10:06   0:01 docker stats container1 container2
vaani    13924   0.0  0.0   4088  2048 pts/4    S+   10:09   0:00 grep --color=auto docker

```

Process monitoring using Docker.

Lab 3: Application Logs

```

Error response from daemon: No such container: c2ecdeeb400d
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker logs c2ecdeeb400d
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/27 09:16:32 [notice] 1#1: using the "epoll" event method
2026/03/27 09:16:32 [notice] 1#1: nginx/1.29.5
2026/03/27 09:16:32 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/03/27 09:16:32 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/27 09:16:32 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/27 09:16:32 [notice] 1#1: start worker processes
2026/03/27 09:16:32 [notice] 1#1: start worker process 28
2026/03/27 09:16:32 [notice] 1#1: start worker process 29
2026/03/27 09:16:32 [notice] 1#1: start worker process 30
2026/03/27 09:16:32 [notice] 1#1: start worker process 31
2026/03/27 09:16:32 [notice] 1#1: start worker process 32
2026/03/27 09:16:32 [notice] 1#1: start worker process 33
2026/03/27 09:16:32 [notice] 1#1: start worker process 34
2026/03/27 09:16:32 [notice] 1#1: start worker process 35
172.17.0.1 - - [27/Mar/2026:09:16:58 +0000] "GET / HTTP/1.1" 200 26 "-" "curl/8.5.0" "-"

Error response from daemon: No such container: 400d
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker logs -f c2ecdeeb400d
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/27 09:16:32 [notice] 1#1: using the "epoll" event method
2026/03/27 09:16:32 [notice] 1#1: nginx/1.29.5
2026/03/27 09:16:32 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/03/27 09:16:32 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/27 09:16:32 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/27 09:16:32 [notice] 1#1: start worker processes
2026/03/27 09:16:32 [notice] 1#1: start worker process 28
2026/03/27 09:16:32 [notice] 1#1: start worker process 29
2026/03/27 09:16:32 [notice] 1#1: start worker process 30
2026/03/27 09:16:32 [notice] 1#1: start worker process 31
2026/03/27 09:16:32 [notice] 1#1: start worker process 32
2026/03/27 09:16:32 [notice] 1#1: start worker process 33
2026/03/27 09:16:32 [notice] 1#1: start worker process 34
2026/03/27 09:16:32 [notice] 1#1: start worker process 35
172.17.0.1 - - [27/Mar/2026:09:16:58 +0000] "GET / HTTP/1.1" 200 26 "-" "curl/8.5.0" "-"

```



```
Error response from daemon: No such container: 400d
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker logs -f c2ecdeeb400d
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/27 09:16:32 [notice] 1#1: using the "epoll" event method
2026/03/27 09:16:32 [notice] 1#1: nginx/1.29.5
2026/03/27 09:16:32 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/03/27 09:16:32 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/27 09:16:32 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/27 09:16:32 [notice] 1#1: start worker processes
2026/03/27 09:16:32 [notice] 1#1: start worker process 28
2026/03/27 09:16:32 [notice] 1#1: start worker process 29
2026/03/27 09:16:32 [notice] 1#1: start worker process 30
2026/03/27 09:16:32 [notice] 1#1: start worker process 31
2026/03/27 09:16:32 [notice] 1#1: start worker process 32
2026/03/27 09:16:32 [notice] 1#1: start worker process 33
2026/03/27 09:16:32 [notice] 1#1: start worker process 34
2026/03/27 09:16:32 [notice] 1#1: start worker process 35
172.17.0.1 - - [27/Mar/2026:09:16:58 +0000] "GET / HTTP/1.1" 200 26 "-" "curl/8.5.0" "-"
```

Viewing container logs.

Lab 4: Container Inspection

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker logs --tail 100 c2ecdeeb400d
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/27 09:16:32 [notice] 1#1: using the "epoll" event method
2026/03/27 09:16:32 [notice] 1#1: nginx/1.29.5
2026/03/27 09:16:32 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/03/27 09:16:32 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/27 09:16:32 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/27 09:16:32 [notice] 1#1: start worker processes
2026/03/27 09:16:32 [notice] 1#1: start worker process 28
2026/03/27 09:16:32 [notice] 1#1: start worker process 29
2026/03/27 09:16:32 [notice] 1#1: start worker process 30
2026/03/27 09:16:32 [notice] 1#1: start worker process 31
2026/03/27 09:16:32 [notice] 1#1: start worker process 32
2026/03/27 09:16:32 [notice] 1#1: start worker process 33
2026/03/27 09:16:32 [notice] 1#1: start worker process 34
2026/03/27 09:16:32 [notice] 1#1: start worker process 35
172.17.0.1 - - [27/Mar/2026:09:16:58 +0000] "GET / HTTP/1.1" 200 26 "-" "curl/8.5.0" "-"

```

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect c2ecdeeb400d
[
  {
    "Id": "c2ecdeeb400d13ca808d2c8b1026ba40c9ea324a12ab05e4c0395b20e8bbc863",
    "Created": "2026-03-27T09:10:53.078383087Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 9982,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-03-27T09:16:31.930670569Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    }
  }
]

```

Inspect container information.

Lab 5 and lab 6

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect --format '{{.State.Status}}' c2ecdeeb400d
running
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect --format '{{.NetworkSettings.IPAddress}}' c2ecdeeb400d
172.17.0.7
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect --format '{{.Config.Env}}' c2ecdeeb400d
[PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin NGINX_VERSION=1.29.5 NJS_VERSION=0.9.5 NJS_RELEASE=1~trixie ACME_V
ERSION=0.3.1 PKG_RELEASE=1~trixie DYNPKG_RELEASE=1~trixie]
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect --format '{{.HostConfig.Memory}}' c2ecdeeb400d
0
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker inspect --format '{{.HostConfig.NanoCpus}}' c2ecdeeb400d
0

```

Monitoring containers.

Advanced monitoring commands.

PART 4: Default Networks

Lab 1

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ nano monitor.sh
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ chmod +x monitor.sh
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ./monitor.sh
=== Docker Monitoring Dashboard ===
Time: Fri Mar 27 10:26:15 UTC 2026

Running Containers:
NAME                STATUS              PORTS
anny_hodgkin        Up 22 minutes      3000/tcp
p3                  Up 23 minutes      0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
inx-custom          Up About an hour   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
w-mysql             Up About an hour   3306/tcp, 33060/tcp
b3                  Up About an hour   80/tcp
b2                  Up About an hour   80/tcp
b1                  Up 2 hours          80/tcp
rtainer             Up 2 hours          0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp,
0000/tcp
d-mycluster-serverlb Up 2 hours          80/tcp, 0.0.0.0:43783->6443/tcp
d-mycluster-server-0 Up 2 hours

Resource Usage:
NAME                CPU %      MEM USAGE / LIMIT   NET I/O      BLOCK I/O
anny_hodgkin        0.03%      15.63MiB / 7.64GiB   1.08kB / 126B 0B / 0B
p3                  0.00%      19.06MiB / 7.64GiB   1.2kB / 126B  49.4MB / 0B
inx-custom          0.00%      8.449MiB / 7.64GiB   2.03kB / 670B  1.62MB / 8.19kB
w-mysql             0.87%      349.5MiB / 7.64GiB   1.64kB / 126B  65.5kB / 15.1MB
b3                  0.00%      7.688MiB / 7.64GiB   1.89kB / 126B  49.2kB / 12.3kB
b2                  0.00%      7.602MiB / 7.64GiB   2.01kB / 126B  0B / 12.3kB
```

Docker network overview.

Lab 2

a) Bridge Network

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
784d3e3863c0	26febclasstask_wordpress-network	bridge	local
1f674c187662	app_net	macvlan	local
4d6502cf6934	bridge	bridge	local
ceb3d370fe5d1	classtask25feb_default	bridge	local
c38f05c560d7	classtask25feb_wordpress-network	bridge	local
248d677675be	docker-postgres-project_app_net	bridge	local
71593a6fcd9e	docker_gwbridge	bridge	local
1fa1fffb054a4	host	host	local
pwhu3vcmhuxg	ingress	overlay	swarm
d275fb8d4e42	k3d-mycluster	bridge	local
efb6d333e417	my_bridge	bridge	local
3522324c3931	my_ipvlan	ipvlan	local
y8qxvlgmyt0	my_overlay	overlay	swarm
9fa9c5fdad26	none	null	local

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
784d3e3863c0       26febclasstask_wordpress-network  bridge             local
1f674c187662       app_net             macvlan             local
4d6502cf6934       bridge              bridge              local
cebd370fe5d1       classtask25feb_default             bridge             local
c38f05c560d7       classtask25feb_wordpress-network   bridge             local
248d677675be       docker-postgres-project_app_net     bridge             local
71593a6fcd9e       docker_gwbridge     bridge              local
1fa1ffb054a4       host                host                local
pwhu3vcmhuxg       ingress             overlay             swarm
d275fb8d4e42       k3d-mycluster       bridge              local
efb6d333e417       my_bridge           bridge              local
3522324c3931       my_ipvlan           ipvlan              local
y8qxxvlgmyt0       my_overlay          overlay             swarm
9fa9c5fdad26       none                null                local
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create my-network
f7e7efa9049ed73baaa16c24ca36ad319da9a9a168348101cc78cf7336c9afbc
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network inspect my-network
[
  {
    "Name": "my-network",
    "Id": "f7e7efa9049ed73baaa16c24ca36ad319da9a9a168348101cc78cf7336c9afbc",
```

Bridge network.

b) Host Network

```

joviat@kare:
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm web1
Error response from daemon: cannot remove container "web1": container is running: stop the container before removing or force remove
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker stop web1
web1
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm web1
web1
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d --name web1 --network my-network nginx
alcb4ec0240633e656714037b083ba35b75d1b10344023f2307af4186c2c79e
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d --name web1 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web1" is already in use by container "alcb4ec0240633e656714037b083ba35b75d1b10344023f2307af4186c2c79e". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker exec web1 curl http://web2
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed

  0     0    0     0    0     0      0      0      0     0  0:00:09  0:00:09  0:00:09  0curl: (6) Could not resolve host: web2
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$

```

Host network.

c) None Network

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d --name host-app --network host nginx
6da40eae29028e868d270b8f0e9ee89dfda2bdea0051ef553aeca8627aa7f8f4
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

None network.

d) Overlay Network

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d --name isolated-app --network none alpine sleep 3600
f550a28abc8d08a2f105e86b01ab3435c2b9e92c495f8abcb65ea836ad4edb21
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker exec isolated-app ifconfig
lo
  Link encap:Local Loopback
  inet addr:127.0.0.1  Mask:255.0.0.0
  inet6 addr: ::1/128 Scope:Host
  UP LOOPBACK RUNNING  MTU:65536  Metric:1
  RX packets:0 errors:0 dropped:0 overruns:0 frame:0
  TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
  collisions:0 txqueuelen:1000
  RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Overlay network.

Lab 3

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create --driver overlay my-overlay
w0tt6srr56atmhga55i2s5t6j
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |
```

Network example.

Lab 4


```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create app-network
e38d038bfed8a4f061f0b4cb6b637fd8a63b6e756bd36dc28b7843e5d45a3691
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create --driver bridge --subnet 172.20.0.0/16 --gateway 172.20.0.1 my
-subnet
Error response from daemon: invalid pool request: Pool overlaps with other one on this address space
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create \
--driver bridge \
--subnet 172.30.0.0/16 \
--gateway 172.30.0.1 \
my-subnet
193cdd84c98afe3dbdeb140e37a309806aef7b8faa4848295fa6dbb215dcce75
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network connect app-network existing-container
Error response from daemon: No such container: existing-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network connect app-network c2ecdeeb400d
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network disconnect app-network c2ecdeeb400d

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create \
--driver bridge \
--subnet 172.30.0.0/16 \
--gateway 172.30.0.1 \
my-subnet
193cdd84c98afe3dbdeb140e37a309806aef7b8faa4848295fa6dbb215dcce75
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network connect app-network existing-container
Error response from daemon: No such container: existing-container
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network connect app-network c2ecdeeb400d
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network disconnect app-network c2ecdeeb400d
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network prune
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] n
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create app-network
Error response from daemon: network with name app-network already exists
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create app-network2
d03a6c985f0d152ceba989110b68480613014b1ff2e6f2f2ef32cba4f91aee1a
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
--name postgres-db \
--network app-network2 \
-e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data \
postgres:15
bab6168ed9ff078168c30bc3967a2535c09828cdaa4d0599d779b4292cc3d3d2

```

Network commands.

Lab 6


```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization$ ls
26FebclassTask  'Experiment 4.docx'  classTask 6Feb  exp3file
'Container project report.docx'  'Project Report DeepScan Minor (2).docx'  classTask26Feb  exp4
'Container project screenshots.docx'  'Project Report DeepScan.docx'  exp 5 files  java-docker
'Containerization class 23jan.docx'  class task 18 Feb  'exp2 C^D.docx'  theory images
'Exp1 C&D.docx'  'class test task'  'exp3 C^D.docx'
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization$ find . -name "package.json"
./exp4/my-node-app/package.json
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization$ cd node-app
-bash: cd: node-app: No such file or directory
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization$ cd exp4/my-node-app
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ ls
Dockerfile app.js package.json
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker build -t node-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM node:18-alpine
--> ee77c6cd7c18
Step 2/7 : WORKDIR /app
--> Using cache
--> e676d7e9ceef
Step 3/7 : COPY package*.json ./
--> Using cache
--> 9f6aa611625d
Step 4/7 : RUN npm install --only=production
--> Using cache
--> ed83bb371bb6
Step 5/7 : COPY app.js .
--> Using cache
--> ac92954bbe5c

```

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker rm -f app1
app1
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker run -d -p 8082:80 --name
app1 nginx
49f5dc4ab12a5545fa55d2272defe3b2860922de935193e6889818752b1d08e4
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
49f5dc4ab12a   nginx         "/docker-entrypoint..." 11 seconds ago Up 10 seconds 0.0.0.0:8082->80/tcp,
[::]:8082->80/tcp
bab6168ed9ff   postgres:15   "docker-entrypoint.s..." 18 minutes ago Up 18 minutes 5432/tcp
f550a28abc8d   alpine        "sleep 3600"             28 minutes ago Up 28 minutes
6da40eae2902   nginx         "/docker-entrypoint..." 29 minutes ago Up 29 minutes
a1cbe4ec0240   nginx         "/docker-entrypoint..." 34 minutes ago Up 34 minutes 80/tcp
b43b7cbe72e9   my-node-app   "docker-entrypoint.s..." About an hour ago Up About an hour 3000/tcp
f45013441b2a   my-node-app   "docker-entrypoint.s..." About an hour ago Up About an hour 0.0.0.0:3000->3000/tc
p, [::]:3000->3000/tcp
c2ecddeb400d   nginx         "/docker-entrypoint..." 2 hours ago   Up 2 hours    0.0.0.0:8080->80/tcp,
[::]:8080->80/tcp
a422968c84a2   mysql:8.0     "docker-entrypoint.s..." 2 hours ago   Up 2 hours    3306/tcp, 33060/tcp
e193e6b7f9a5   nginx         "/docker-entrypoint..." 2 hours ago   Up 2 hours    80/tcp
01bb6ab46a36   nginx         "/docker-entrypoint..." 2 hours ago   Up 2 hours    80/tcp
2ecff23c2c7d   portainer/portainer-ce:lt
s "/portainer"             7 days ago    Up 3 hours    0.0.0.0:8000->8000/tc

```

Advanced networking.

PART 5

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker network create myapp-network
2b735154a91de9f7933df681b41d423fba8632720806e6f1619e740e086db2f0
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker run -d \
  --name postgres \
  --network myapp-network \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_DB=mydatabase \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:15
3a31d47ddcd644f764f72400faf9f38b1ebdc8b060e44c26291168e712da0415
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker run -d \
  --name redis \
  --network myapp-network \
  -v redis-data:/data \
  redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
bc1da058f299: Pull complete
09a5a0c32a23: Pull complete
bd53938b1271: Pull complete
9fe3744a2eac: Pull complete
7ad54b3c4cef: Pull complete
2a97533d89ca: Pull complete
4f4fb700ef54: Pull complete
1c2c8d1ee428: Pull complete
Digest: sha256:8b81dd37ff027bec4e516d41acfb9fe2460070dc6d4a4570a2ac5b9d59df065
Status: Downloaded newer image for redis:7-alpine
2c83f42cb4c6792b1dfcc14b59ff45f11382b772802e6ded528995deb038f3fc
```

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app$ docker run -d \
  --name flask-app \
  --network myapp-network \
  -p 5000:5000 \
  -v "/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp4/my-node-app/app:/app" \
  -v app-logs:/var/log/app \
  -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" \
  -e DEBUG="false" \
  -e LOG_LEVEL="INFO" \
  --env-file .env.production \
  flask-app:latest
c9eab38d2f7cdb6731a3f43e55eaf2e524865d17db1212bf680b7f9e26fac7e
```

Final implementation.

containerization-and-devops

Experiment 6

Part A

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d --name my-nginx -p 8080:80 -v ./html:/usr/share/nginx/html -e
NGINX_HOST=localhost --restart unless-stopped nginx:alpine
488f15b1ca2b04ca14184e133bea8138d4bfbfd3fb78d136d27323c9b0cca679
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$
```

Part B

Task 1: Run nginx using docker run

Output Screenshot

```
run docker run --help for more information
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
--name lab-nginx \
-p 8081:80 \
-v "$(pwd)/html:/usr/share/nginx/html" \
nginx:alpine
db5dad5bd4289b518bec386d40abf1645616f2a9ea4457328302af5723cecc96
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	NAMES	STATUS	PORTS
db5dad5bd428	nginx:alpine	"/docker-entrypoint..."	5 seconds ago	lab-nginx	Up 4 seconds	0.0.0.0:8081->80/tcp
488f15b1ca2b	nginx:alpine	"/docker-entrypoint..."	About a minute ago	my-nginx	Up About a minute	0.0.0.0:8080->80/tcp
2ecff23c2c7d	portainer/portainer-ce:latest	"/portainer"	2 weeks ago	portainer	Up About a minute	0.0.0.0:8000->8000/tcp
f1c8e59c370b	ghcr.io/k3d-io/k3d-proxy:5.8.3	"/bin/sh -c nginx-pr..."	2 weeks ago	k3d-mycluster-serverlb	Up About a minute	80/tcp, 0.0.0.0:437
83->6443/tcp						
fc23f654b650	rancher/k3s:v1.31.5-k3s1	"/bin/k3d-entrypoint..."	2 weeks ago	k3d-mycluster-server-0	Up About a minute	

```

← → ↻ ⓘ http://localhost:8081
Vidyamandir History Downloads New Tab mind map
```

403 Forbidden

nginx/1.29.5

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ nano docker-compose.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ mkdir html
mkdir: cannot create directory 'html': File exists
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ls html
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ nano html/index.html
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use 'docker stack deploy'.

Creating network "vaanijindal_default" with the default driver
Creating lab-nginx ... done
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
3fd4050fd486	nginx:alpine	"/docker-entrypoint..."	8 seconds ago	Up 7 seconds	0.0.0.0:8081->80/tcp, [::]:80
81->80/tcp					
488f15b1ca2b	nginx:alpine	"/docker-entrypoint..."	7 minutes ago	Up 7 minutes	0.0.0.0:8080->80/tcp, [::]:80
80->80/tcp					
2ecff23c2c7d	portainer/portainer-ce:lts	"/portainer"	2 weeks ago	Up 8 minutes	0.0.0.0:8000->8000/tcp, [::]:
8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 9000/tcp					
f1c8e59c370b	ghcr.io/k3d-io/k3d-proxy:5.8.3	"/bin/sh -c nginx-pr..."	2 weeks ago	Up 8 minutes	80/tcp, 0.0.0.0:43783->6443/t
cp					
fc23f654b650	rancher/k3s:v1.31.5-k3s1	"/bin/k3d-entrypoint..."	2 weeks ago	Up 8 minutes	k3d-mycluster-server-0

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |

```

← → ↻ ⓘ http://localhost:8081

🗄️ | 🖼️ Vidyamandir 🔍 History ⬇️ Downloads New Tab 📺 mind map

Docker Compose Nginx Working ðŸš€

Lab test successful

Task 2: Multi Container Application

Create Network

Run Container 1

Run Container 2

Screenshots


```

Run 'docker run --help' for more information
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker network create wp-net
3b3493fe8c598dc0e979ca956451d1ac3bb0ae5c85ee4aff0ce6cb134f1f6111
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
--name mysql \
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
docker: Error response from daemon: Conflict. The container name "/mysql" is already in use by container "cb85a00a6cfe50c5a41f4eb7805912be4c94ec57362f709e79185a5dd2c363e". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker ps -a

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
3fd4050fd486	nginx:alpine	"/docker-entrypoint..."	2 minutes ago	Up 2 minutes	0.0.0.0:80
81->80/tcp, [::]:8081->80/tcp				lab-nginx	
488f15b1ca2b	nginx:alpine	"/docker-entrypoint..."	10 minutes ago	Up 10 minutes	0.0.0.0:80
80->80/tcp, [::]:8080->80/tcp				my-nginx	
0c9eab38d2f7	flask-app:latest	"docker-entrypoint.s..."	11 days ago	Exited (1) 11 days ago	
2c83f42cb4c6	redis:7-alpine	"docker-entrypoint.s..."	11 days ago	Exited (255) 10 minutes ago	6379/tcp
3a31d47ddcd6	postgres:15	"docker-entrypoint.s..."	11 days ago	Exited (255) 10 minutes ago	5432/tcp
49f5dc4ab12a	nginx	"/docker-entrypoint..."	11 days ago	Exited (255) 10 minutes ago	0.0.0.0:80

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm mysql
mysql
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
--name mysql \
--network wp-net \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
9c65b2c3c226df8e1d0bb3c85bc9028f25e6838117207f8d160734e53fbe0ec2
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
--name wordpress \
--network wp-net \
-p 8082:80 \
-e WORDPRESS_DB_HOST=mysql \
-e WORDPRESS_DB_PASSWORD=secret \
wordpress:latest
07663b5cf621f95ad10acee08be5798985519bfff162eea4eb0d817d896ea4c90
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |

```

← → ↻ ⓘ http://localhost:8082 ☆

📁 Vidyamandir 🕒 History 📁 Downloads 🆕 New Tab 📺 mind map

Error establishing a database connection

Part C

Task 3: Convert docker run to docker compose

docker-compose.yml

```

version: '3'
services:
  nginx:
    image: nginx
    ports:
      - "8080:80"

```

Run Command

```
docker-compose up -d
```

Screenshot

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ nano docker-compose.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
  --name webapp \
  -p 5000:5000 \
  -e APP_ENV=production \
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine
023812fd0cfb7d677890628eccf317882fb772687f25228c738e04b4f1820f30

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ nano docker-compose.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All
.

To deploy your application across the swarm, use `docker stack deploy`.

Creating volume "vaanijindal_mysql_data" with default driver
WARNING: Found orphan containers (lab-nginx) for this project. If you removed or re
un this command with the --remove-orphans flag to clean it up.
Creating mysql ... error

ERROR: for mysql Cannot create container for service mysql: Conflict. The containe
65b2c3c226df8e1d0bb3c85bc9028f25e6838117207f8d160734e53fbe0ec2". You have to remove
at name.

ERROR: for mysql Cannot create container for service mysql: Conflict. The containe
65b2c3c226df8e1d0bb3c85bc9028f25e6838117207f8d160734e53fbe0ec2". You have to remove
at name.
ERROR: Encountered errors while bringing up the project.
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm -f mysql
mysql

```

```

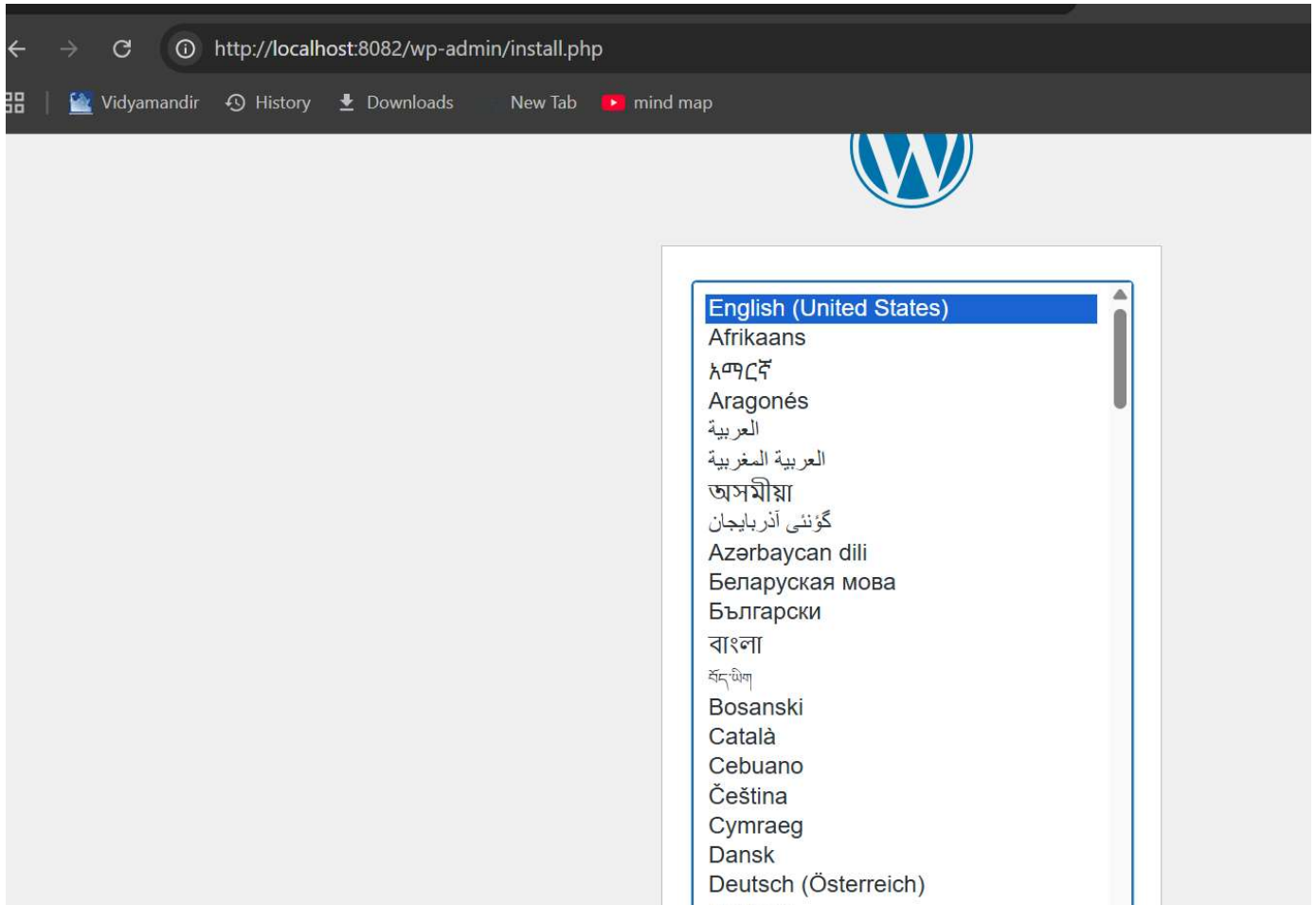
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker rm -f wordpress
wordpress
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the same node.

To deploy your application across the swarm, use 'docker stack deploy'.

mysql is up-to-date
Creating wordpress ... done
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |

```



Task 4: Resource Limit Conversion

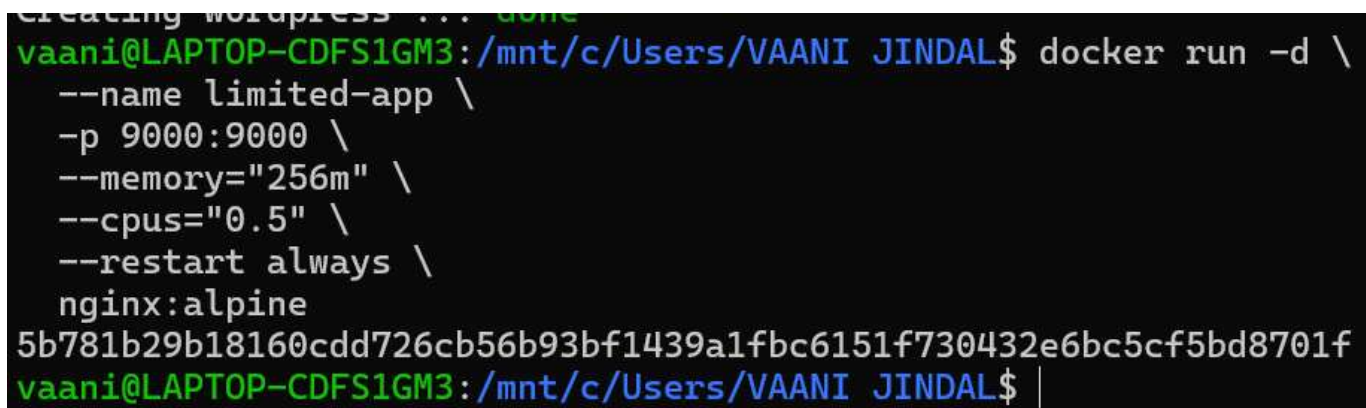
docker-compose.yml

```

version: '3'
services:
  nginx:
    image: nginx
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 512M

```

Screenshot



```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \
  --restart always \
  nginx:alpine
5b781b29b18160cdd726cb56b93bf1439a1fbc6151f730432e6bc5cf5bd8701f
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ |
```

Part D : Using Dockerfile Instead of Image

Task 5

Dockerfile

```
FROM nginx
COPY index.html /usr/share/nginx/html/index.html
```

Build Image

```
docker build -t custom-nginx .
```

Run Container

```
docker run -d -p 8080:80 custom-nginx
```

Screenshot


```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ mkdir node-app
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ cd node-app
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ nano app.js
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ nano Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ nano docker-compose.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ docker-compose up -d --build
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All con
.

To deploy your application across the swarm, use `docker stack deploy`.

Creating network "node-app_default" with the default driver
Building nodeapp
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

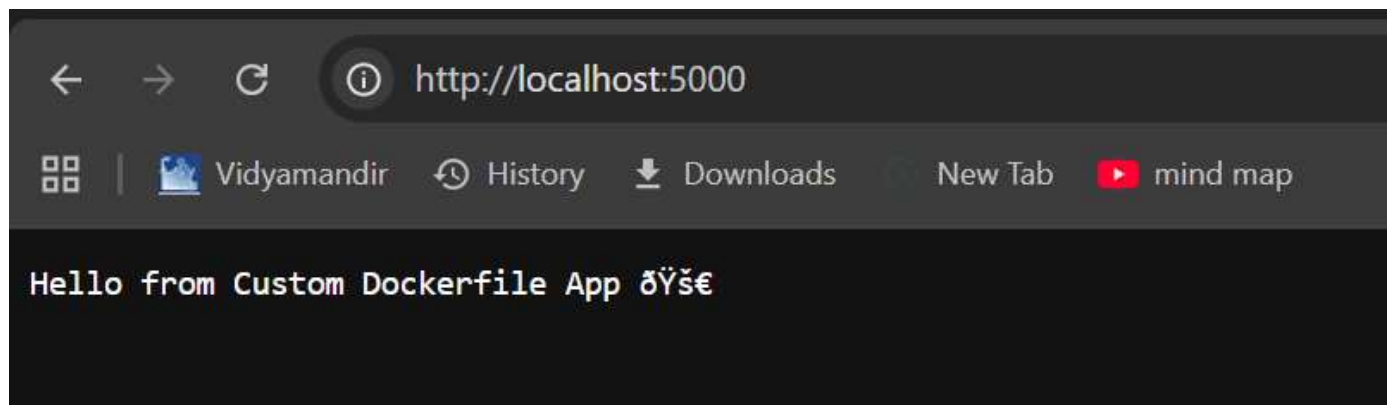
Sending build context to Docker daemon  4.096kB
Step 1/5 : FROM node:18-alpine
----> ee77c6cd7c18
Step 2/5 : WORKDIR /app
----> Using cache
----> e676d7e9ceef
Step 3/5 : COPY app.js .
----> f8e82b57b6ba
Step 4/5 : EXPOSE 5000
----> Running in 355ffcd69964
----> Removed intermediate container 355ffcd69964
----> 4a7328b56faf
Step 5/5 : CMD ["node", "app.js"]
----> Running in 6b1b05a10fb5

```

```

Step 4/5 : EXPOSE 5000
----> Running in 355ffcd69964
----> Removed intermediate container 355ffcd69964
----> 4a7328b56faf
Step 5/5 : CMD ["node", "app.js"]
----> Running in 6b1b05a10fb5
----> Removed intermediate container 6b1b05a10fb5
----> 02d343fcd332
Successfully built 02d343fcd332
Successfully tagged node-app_nodeapp:latest
Creating custom-node-app ... done
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS      PORTS
fa9b2690648f   node-app_nodeapp                    "docker-entrypoint.s..." 8 seconds ago  Up 8 seconds    0.0.0.0:5000

```



Task 6: Multistage Dockerfile with Compose

Multi-stage Dockerfile

```
FROM node:16 as build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

FROM nginx

COPY --from=build /app/build /usr/share/nginx/html
```

docker-compose.yml

```
version: '3'

services:
  web:
    build: .
    ports:
      - "8080:80"
```

Run

```
docker-compose up --build
```

Screenshots

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ mkdir multi-stage-app
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app$ cd multi-stage-app
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app/multi-stage-app$ nano server.js
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app/multi-stage-app$ nano package.json
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app/multi-stage-app$ nano Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app/multi-stage-app$ nano docker-compose.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app/multi-stage-app$ docker-compose up -d --build
WARNING: The Docker Engine you're using is running in swarm mode.

```

Docker Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be

to deploy your application across the swarm, use 'docker stack deploy'.

Creating network "multi-stage-app_default" with the default driver

Building multiapp

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

Install the buildx component to build images with BuildKit:

<https://docs.docker.com/go/buildx/>

Sending build context to Docker daemon 5.12kB

Step 1/10 : FROM node:18-alpine AS builder

----> ee77c6cd7c18

Step 2/10 : WORKDIR /app

----> Using cache

----> e676d7e9ceef

Step 3/10 : COPY package.json .

----> 63c1e3822ae3

Step 4/10 : RUN npm install

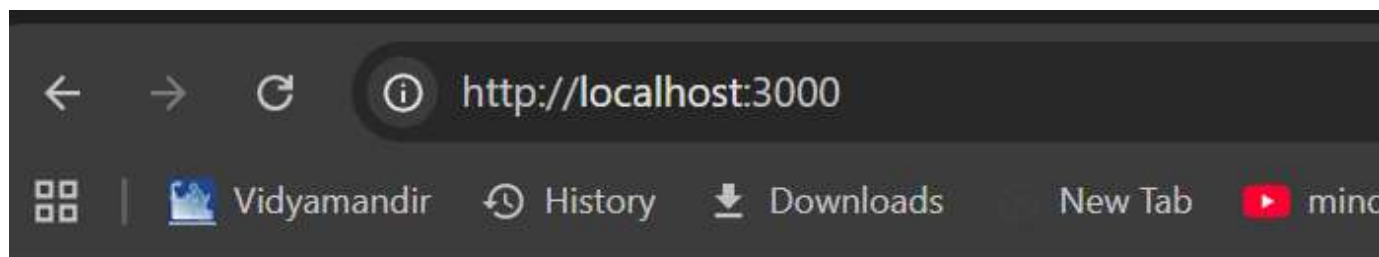
----> Running in 5e87002d2725

added 68 packages, and audited 69 packages in 3s

```

npm notice
----> Removed intermediate container 5e87002d2725
----> 095512988cb6
Step 5/10 : COPY . .
----> ff2a7d5bb0eb
Step 6/10 : FROM node:18-alpine
----> ee77c6cd7c18
Step 7/10 : WORKDIR /app
----> Using cache
----> e676d7e9ceef
Step 8/10 : COPY --from=builder /app .
----> 96a2b6c7da34
Step 9/10 : EXPOSE 3000
----> Running in 749da492a083
----> Removed intermediate container 749da492a083
----> 21ad19cc7d8f
Step 10/10 : CMD ["node", "server.js"]
----> Running in fafb740998ba
----> Removed intermediate container fafb740998ba
----> cc6aa83652ad
Successfully built cc6aa83652ad
Successfully tagged multi-stage-app_multiapp:latest
Creating multi-stage-app ... done
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/node-app/multi-stage-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
ed9bacbdd5bb   multi-stage-app_multiapp            "docker-entrypoint.s..." 9 seconds ago    Up 8 seconds    0.0.0.0:3000->3000/tcp, [:::3000->3000/tcp]

```



Multi-Stage Docker App Running 🚀

Experiment 6B

Multi Container Application using Docker Compose

docker-compose.yml

```
version: '3'

services:
  web:
    image: nginx
    ports:
      - "8080:80"

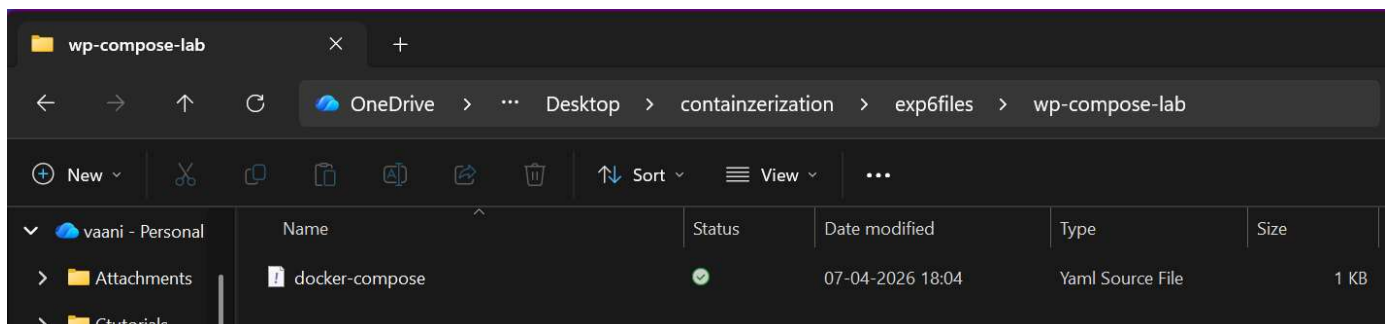
  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
```

Run

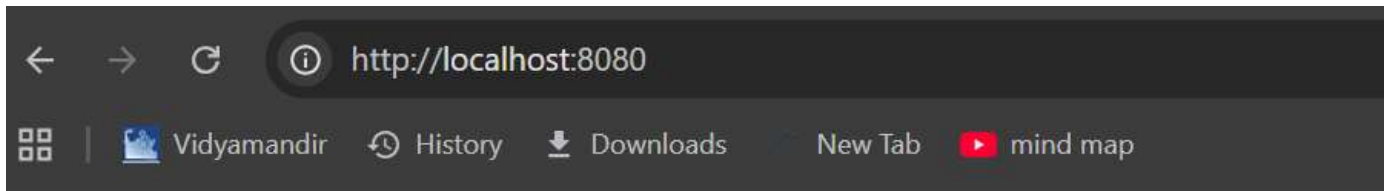
```
docker-compose up -d
```

Screenshot

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp6files> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files$ mkdir wp-compose-lab
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files$ cd wp-compose-lab
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ |
```



!



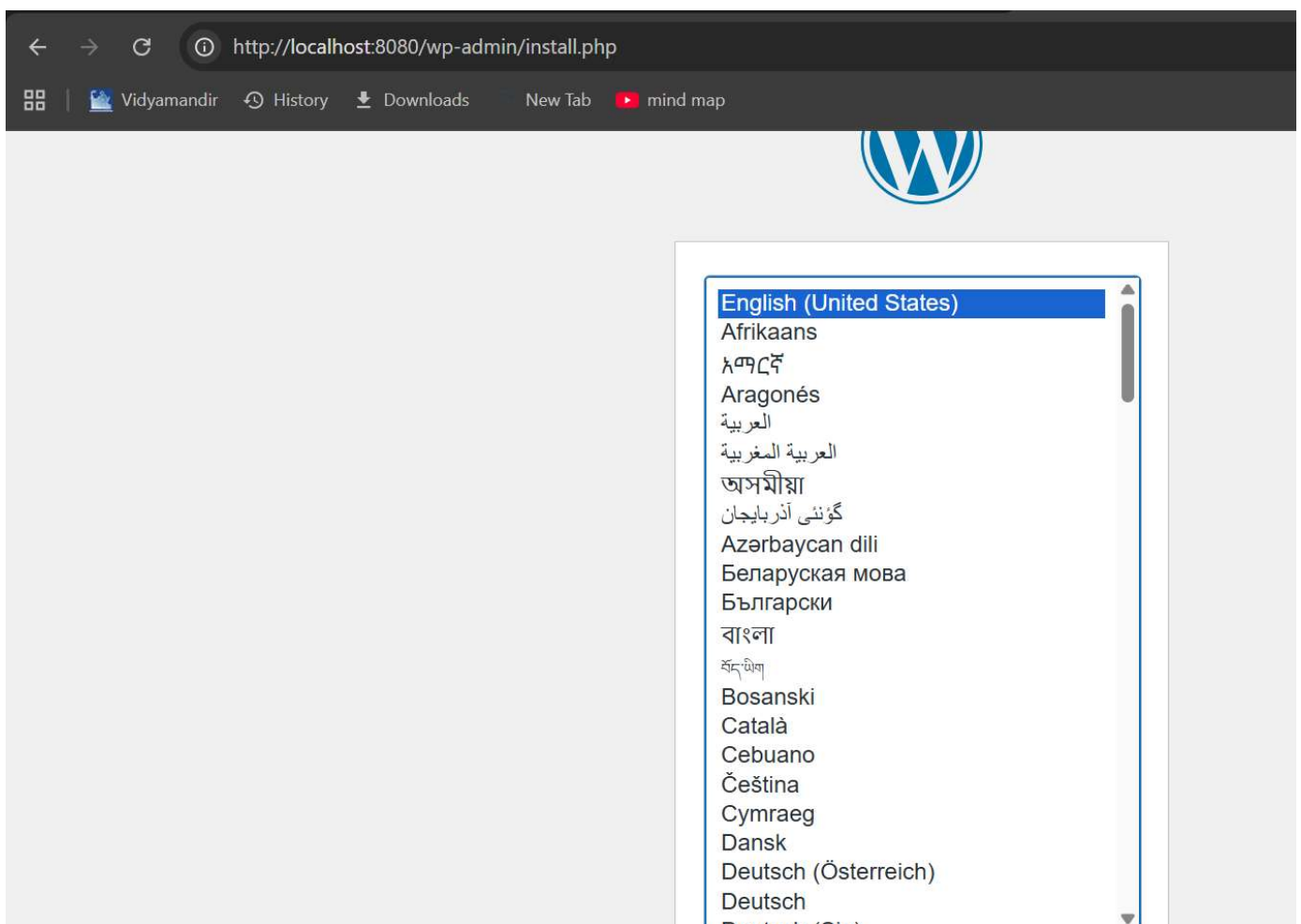
Docker Compose Nginx Working ðŸš€

Lab test successful

Task1 Output

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
f0eb477e7505	wordpress:latest	"docker-entrypoint.s..."	3 minutes ago	Up About a minute	0.0.0.0:8080->80/tcp, [
1dcc8e07c384	mysql:5.7	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	3306/tcp, 33060/tcp
ed9bacbdd5bb	multi-stage-app_multiapp	"docker-entrypoint.s..."	14 minutes ago	Up 14 minutes	0.0.0.0:3000->3000/tcp,



```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ docker volume ls
DRIVER      VOLUME NAME
local       0a6ab9d1dfc129c1700caf6b9f55d671346ea7d9ba9ce484d14ad64089cb3007
local       0d86627c85ce8c4e5c891bf0a5f8710835ed9366ea8b8aacf81537eb70b886b4
local       0e6c339a392d4bc0707750699b2a6a1e6763bfa3ac7c4d06b9f21662c20b1c79
local       16b9d11543d382153f5059b346db3eda988b294661e47d2ab69a1daf0ca75bc4
local       26febclasstask_mysql_data
local       26febclasstask_wp_content
local       55b372250c1a88ff54acc91c56eefc611ed1224f5073a69943d76414a2cd70a
local       80f158dea58688cfdf7a2cec4edb6130e7f332aa5132570ad0fd3c1ae4739f6c7
local       573e0f8b12cbc2a0eb08dfccea233c022dd633f66de7f056780aa173b6cc547b
local       587a13f0868637dc82ab0448bd380bd469e395c1d8d22f45d37c0e1e848146f1
local       6868def67db12335391b3f12069f9e5bbe43a4aa4a209975c7be8d4b5608beb9
local       a39c6a7fdec1bcc893e84ec7bbe8e0363b60849376531ef79896d6747478e02f
local       app-logs
local       app-volume
local       b7722e82e65d3bb9714c6e4ec6a8b16a188f448178bbfb4642e82ed6aba584fe
local       c830fdb972d39bb08d81d6f15ad4362db939e1452e9d7caeb0882f679d228ec8
local       classtask25feb_mysql_data
local       classtask25feb_wp_content
local       da80498974262dd88c092458cf498b778d25ca89f38496e5da10acc46b899587
local       docker-postgres-project_postgres_data
local       e66c8a4683852864b24f0ff9d20b14a2083233d2a84a4cea2baafe3db15de984
local       k3d-mycluster-images

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ docker-compose down
Stopping wordpress_app ... done
Stopping wordpress_db   ... done
Removing wordpress_app  ... done
Removing wordpress_db   ... done
Removing network wp-compose-lab_default

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ docker swarm init
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join another one.
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
Creating service wpstack_db
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp6files/wp-compose-lab$ S

```

Result

Successfully performed:

- Running nginx using docker run
- Multi container application
- Converting docker run to docker compose
- Resource limit conversion
- Using Dockerfile instead of image
- Multi-stage Dockerfile with compose
- Multi container application using docker compose

Experiment completed successfully.

containerization-and-devops

Experiment 7

Part A:

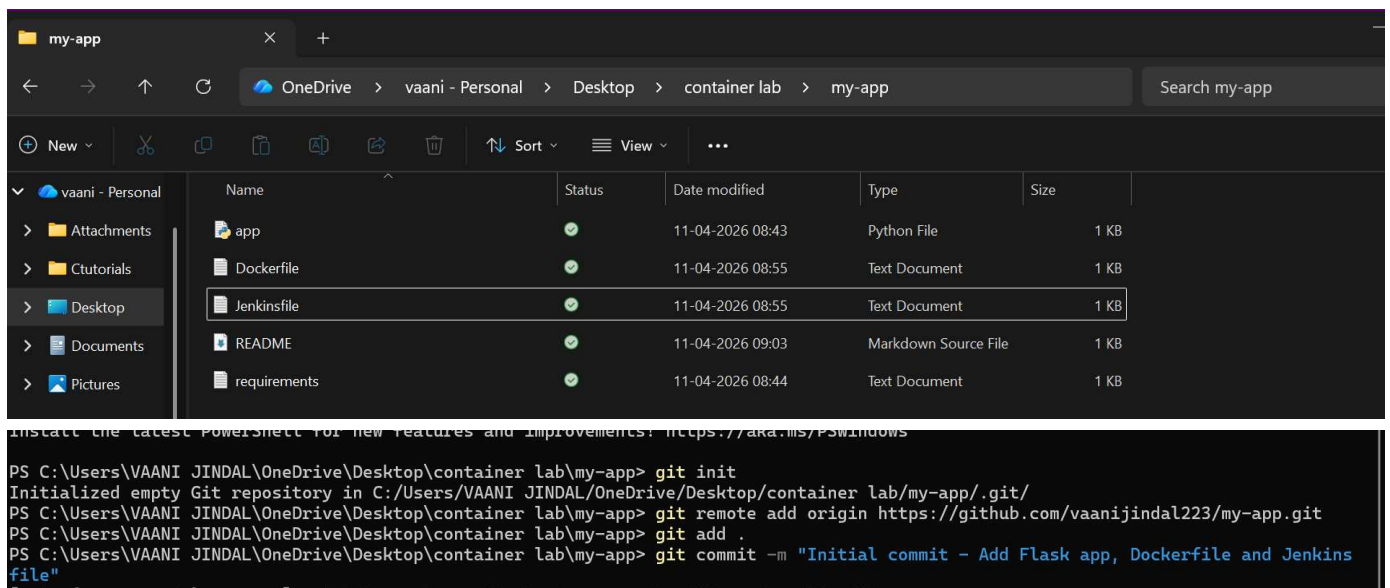
In this part, a CI/CD pipeline setup was planned using GitHub repository for storing application source code.

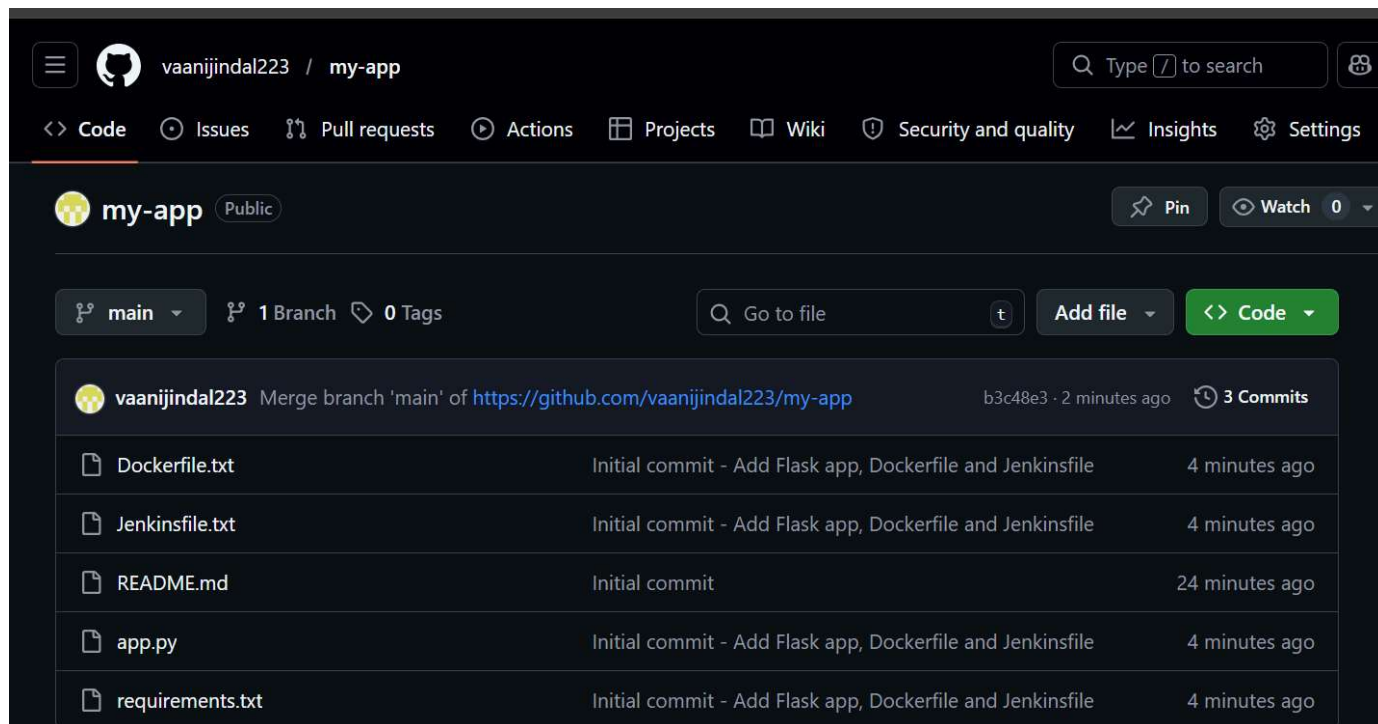
A Flask based application was prepared and pushed to GitHub repository for further automation using Jenkins.

The repository was created and project files were uploaded successfully.

The application structure was verified and repository connection was confirmed.

Screenshots





Part B:

In this part, Docker containerization was implemented for the application.

Docker image was created using Dockerfile and application dependencies were installed.

Docker container was executed successfully and application was verified using browser.

Docker environment setup was completed successfully and container execution was verified.

Screenshots


```

version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  jenkins_home:

```

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab/my-app$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use 'docker stack deploy'.

```

```

Creating network "my-app_default" with the default driver
Creating volume "my-app_jenkins_home" with default driver
Pulling jenkins (jenkins/jenkins:lts)...
lts: Pulling from jenkins/jenkins
53c88f1dfeb7: Pull complete
4d2125ccf71c: Pull complete
69503609dc01: Pull complete
6da7907f70f4: Pull complete
6327a7366a6f: Pull complete
01938f7c040b: Pull complete
b7b154fd69a1: Pull complete
876217fdee20: Pull complete
fbae626c1b7a: Pull complete
b19825c62ed8: Pull complete
ec7478903548: Pull complete
e682a95ad036: Pull complete
Digest: sha256:c4098086090ca98491d4bf66182f5e3b015a8232f2acf2df209a212a5801aa8e
Status: Downloaded newer image for jenkins/jenkins:lts
Creating jenkins ... done
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab/my-app$

```

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab/my-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
6fda4535a620   jenkins/jenkins:lts                "/usr/bin/tini -- /u..." 3 minutes ago  Up 3 minutes  0.0.0.0:8080->8080/t
cp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
jenkins
f6f1e7ba3b8d   docker-postgres-project_backend     "docker-entrypoint.s..." About an hour ago  Up About an hour  0.0.0.0:3000->3000/t
cp, [::]:3000->3000/tcp
backend_container
a4ae0db15f86   docker-postgres-project_db          "docker-entrypoint.s..." About an hour ago  Up About an hour  5432/tcp
postgres_container
885c4dfa47f0   wordpress:latest                   "docker-entrypoint.s..." About an hour ago  Up About an hour  80/tcp
wpstack_wordpress.1.9nl96vv67ddgt62jjotqu62lz
a6f23bd8acd6   mysql:5.7                           "docker-entrypoint.s..." About an hour ago  Up About an hour  3306/tcp, 33060/tcp
wpstack_db.1.ptcgmajeye53v3neflzgp3h19
432626649bb4   wordpress:latest                   "docker-entrypoint.s..." About an hour ago  Up About an hour  80/tcp
wpstack_wordpress.2.38ielivdszo04z9ajjv9x0eu1
8a4052346519   wordpress:latest                   "docker-entrypoint.s..." About an hour ago  Up About an hour  80/tcp
wpstack_wordpress.3.dcm1xcb24d208lnc6a96ym36k

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab/my-app$ docker exec -it jenkins cat /var/jenkins_home/
secrets/initialAdminPassword
f58a8e49e367447ab4d9716c37f35217
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/container lab/my-app$

```

Getting Started

Jenkins is ready!

Your Jenkins setup is complete.

[Start using Jenkins](#)

New Item

Enter an item name

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

<https://github.com/vaanijindal223/my-app.git>

Credentials ?

Part C:

In this part, Jenkins pipeline configuration was created.
Jenkinsfile was prepared for automating CI/CD workflow.

Pipeline stages included:

- Clone Source
- Build Docker Image
- Login to DockerHub
- Push Docker Image

Jenkins pipeline job was created and GitHub repository was connected successfully.

Screenshots

< Add Secret text ×

Scope ?

Global (Jenkins, nodes, items, all child items, etc) ✓

Secret

.....

ID ?

dockerhub-token

Description ?

DockerHub Credentials

Create

[Personal access tokens](#) / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description

Jenkins

Expiration date

None

Optional

Access permissions

Read & Write

Read & Write tokens allow you to push images to any repository managed by your account.

[Cancel](#) [Generate](#)

Part D:

In this part, Jenkins server was deployed using Docker container.
Jenkins container was started successfully and accessed using browser.

Jenkins initial setup was completed and required plugins were installed.

Docker socket was mounted to allow Jenkins to access Docker daemon.

Jenkins environment setup was completed successfully.

Screenshots

```
4ac66e390851: Pushed
e35db0322204: Mounted from library/python
60e70ddddd9ea: Mounted from library/python
9179a905c29c: Pushed
latest: digest: sha256:6615a4054a6e3cc6a612f4d8ee7507920837fa215b4be5aac03725d5e7cdd2bd size: 1785
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Part E:

In this part, DockerHub credentials were configured inside Jenkins.
Pipeline job execution was triggered and CI/CD workflow executed successfully.

Pipeline executed following stages:

- Clone Source
- Build Docker Image
- Login DockerHub
- Push Docker Image

Docker image was successfully pushed to DockerHub repository.

Screenshots

The image shows two screenshots of GitHub and Docker Hub interfaces. The top screenshot displays the 'Repositories' page for the 'vaanijindal22' namespace, listing three repositories: 'my-app', 'my-flask-app', and 'javaapp'. The bottom screenshot shows the 'Tags' page for the 'my-app' repository on Docker Hub, displaying a table of tags with columns for Digest, OS/ARCH, Last pull, and Compressed size.

Repositories

All repositories within the **vaanijindal22** namespace.

Search by repository name | All content | Create a repository

Name	Last Pushed ↑	Contains	Visibility	Scout
vaanijindal22/my-app	3 minutes ago	IMAGE	Public	Inactive
vaanijindal22/my-flask-app	about 2 months ago	IMAGE	Public	Inactive
vaanijindal22/javaapp	2 months ago	IMAGE	Public	Inactive

Docker Hub: my-app

Last pushed 2 minutes ago · ☆ 0 · ↓ 0

Add a description | Add a category

To push a new tag to this repository:
`docker push vaanijindal22/my-app:tagname`

General | **Tags** | Image Management | Collaborators | Webhooks | Settings

Sort by: Newest | Filter tags | Delete

Digest	OS/ARCH	Last pull	Compressed size ⓘ
6615a4054a6e	linux/amd64	less than 1 day	47.39 MB

Tags: latest (Last pushed 2 minutes by vaanijindal22)

To pull the latest tag:
`docker pull vaanijindal22/my-app:latest`

Result

CI/CD pipeline was successfully implemented using Jenkins, Docker and GitHub.
Application build and deployment process was automated successfully.

Conclusion

The experiment successfully demonstrated implementation of CI/CD pipeline using Jenkins and Docker.
Automation of build and deployment process improved efficiency and reduced manual effort.

containerization-and-devops

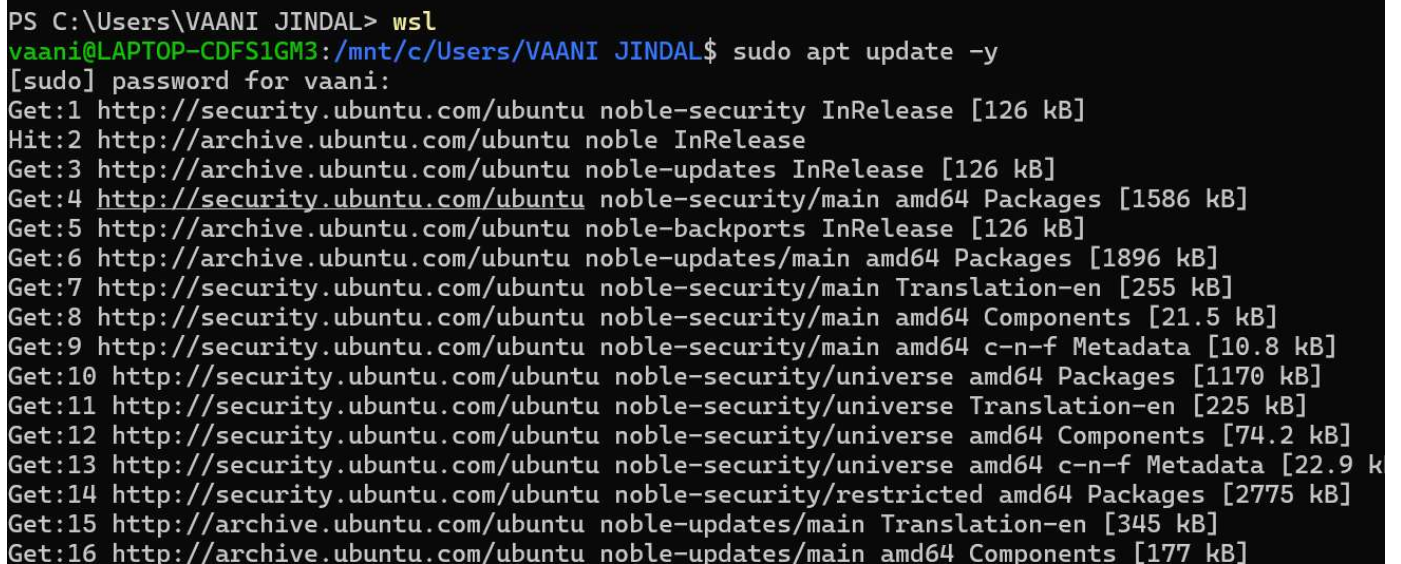
EXPERIMENT 9 C&D

Part A : ansible installations instructions:

Step 1: Update System Packages

```
sudo apt update && sudo apt upgrade -y
```

 Screenshot:



```
PS C:\Users\VAANI JINDAL> wsl
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo apt update -y
[sudo] password for vaani:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1586 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1896 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [255 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.8 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1170 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [225 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [22.9 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2775 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [345 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
```

Step 2: Install Ansible

```
sudo apt install ansible -y
```


 Screenshot:

```

07 packages can be upgraded. Run 'apt list --upgradable' to see them.
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libllvm19 libwayland-server0
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  ansible-core ieee-data python3-argcomplete python3-dnspython python3-jmespath python3-kerberos python3-libcloud
  python3-lockfile python3-netaddr python3-ntlm-auth python3-passlib python3-requests-ntlm python3-resolvelib
  python3-selinux python3-simplejson python3-winrm python3-xmltodict
Suggested packages:
  cowsay sshpass python3-trio python3-aiquic python3-h2 python3-httpx python3-httpcore python-lockfile-doc ipython3
  python-netaddr-docs
The following NEW packages will be installed:
  ansible ansible-core ieee-data python3-argcomplete python3-dnspython python3-jmespath python3-kerberos
  python3-libcloud python3-lockfile python3-netaddr python3-ntlm-auth python3-passlib python3-requests-ntlm
  python3-resolvelib python3-selinux python3-simplejson python3-winrm python3-xmltodict
0 upgraded, 18 newly installed, 0 to remove and 57 not upgraded.

```

Step 3: Verify Installation

```
ansible --version
```

 Screenshot:

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/vaani/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/vaani/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True

```

post installation check :

Ping Localhost

```
ansible localhost -m ping
```

 Screenshot:

```

libyaml = True
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$

```

Part B: Setup for Ansible with Docker (SSH Servers)

1. Create SSH Key Pair in WSL

```
ssh-keygen -t rsa
```

 Screenshot:

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/vaani/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/vaani/.ssh/id_rsa
Your public key has been saved in /home/vaani/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:rnVkJHt02UAPrYqAvFhenGZO3HWgtNZM/HQU8DKA/po vaani@LAPTOP-CDFS1GM3
The key's randomart image is:
+---[RSA 4096]-----+
|      .o+o+=+o.      |
|      ..*o.oBo       |
|      . + +=.=+=o+    |
|      + X.o= .oo      |
|      + B .So+.       |
|      . o ...+o       |
|      oo.             |
|      oE.             |
|      .               |
+-----[SHA256]-----+
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ cp ~/.ssh/id_rsa.pub .
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL$ cp ~/.ssh/id_rsa .
```

Run Docker Containers (SSH Servers)

```
docker run -d -p 2222:22 --name server1 rastasheep/ubuntu-sshd
docker run -d -p 2223:22 --name server2 rastasheep/ubuntu-sshd
```

 Screenshot:

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ docker build -t ansible-ubuntu .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

unable to prepare context: unable to evaluate symlinks in Dockerfile path: lstat /mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files/Dockerfile: no such file or directory
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ls
Dockerfile.txt
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ mv Dockerfile.txt Dockerfile
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ls
Dockerfile
```

Copy SSH Key to Containers

```
ssh-copy-id -p 2222 root@localhost
ssh-copy-id -p 2223 root@localhost
```

 Screenshot:

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ cp ~/.ssh/id_rsa.pub "/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files/"
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ cp ~/.ssh/id_rsa "/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files/"
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ cd "/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files"
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ls
Dockerfile id_rsa id_rsa.pub

```

Create Inventory File

```
[servers]
```

```
server1 ansible_host=127.0.0.1 ansible_port=2222 ansible_user=root
```

```
server2 ansible_host=127.0.0.1 ansible_port=2223 ansible_user=root
```

 Screenshot:

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ docker build -t ansible-ubuntu .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 8.192kB
Step 1/10 : FROM ubuntu
--> 493218ed0f40
Step 2/10 : RUN apt update -y
--> Using cache
--> 4857783ecb1c
Step 3/10 : RUN apt install -y python3 python3-pip openssh-server sudo
--> Using cache
--> ceb3dbed7f44
Step 4/10 : RUN mkdir -p /var/run/ssh
--> Using cache
--> 72ce6b83a563
Step 5/10 : RUN mkdir -p /root/.ssh
--> Using cache
--> 1abb0b01d3ce
Step 6/10 : COPY id_rsa.pub /root/.ssh/authorized_keys
--> c7f9acfeba10
Step 7/10 : RUN chmod 600 /root/.ssh/authorized_keys
--> Running in d1762a225658
--> Removed intermediate container d1762a225658
--> 94d12deb63eb
Step 8/10 : RUN sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && sed -i 's/#PasswordAut

```

Test Ansible Connectivity

```
ansible -i inventory.ini all -m ping
```

 Screenshot:

```

--> 86e90a8f3f54
Successfully built 86e90a8f3f54
Successfully tagged ansible-ubuntu:latest
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ docker run -d -p 2222:22 --name ubuntu
1 ansible-ubuntu
9d659942729f29786f0bcd2d66d32d5a634b019972ead3f996311ade04a8c88f
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$

```

Run Playbook

```
ansible-playbook -i inventory.ini playbook.yml
```

 Screenshot:

```
vaani@LAPTOP-CDFS1GM3: /mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ for i in {1..4}; do
  docker run -d -p 220${i}:22 --name server${i} ansible-ubuntu
done
4898927aa59a2db73ec8d30e55b8fcf25093c82b9265972049d668dd12da1280
81d437bf22bc0fc9ad13c5ec5773a350974259d3ba17b5494fc2e8c0487c5212
1cc4ab8c1e45b9b3f784a5e9f5fa5050e87b9729171d151011f652297b9ee8ce
a94a0485fd36734d72708ba45641b0c50e44e2b703ed6f8ea025938217d2e236
vaani@LAPTOP-CDFS1GM3: /mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ nano inventory.ini
vaani@LAPTOP-CDFS1GM3: /mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ nano inventory.ini
vaani@LAPTOP-CDFS1GM3: /mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ansible all -i inventory.ini -m ping
```

(Multiple Servers + Ansible)

- Multiple Docker containers are used as managed nodes.
- Ansible connects via SSH using generated keys.
- Tasks are executed simultaneously across all servers.
- Ensures automation and scalability.

 Screenshot:

```
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
server2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
yes
server1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
yes
server3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
yes
server4 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```



```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ nano playbook.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ls
Dockerfile id_rsa id_rsa.pub inventory.ini playbook.yml
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ansible-playbook -i inventory.ini play
book.yml

PLAY [Update and configure servers] *****

TASK [Gathering Facts] *****
ok: [server3]
ok: [server1]
ok: [server2]
ok: [server4]

TASK [Update apt packages] *****
ok: [server3]
ok: [server4]
ok: [server1]
ok: [server2]

TASK [Install packages] *****
changed: [server1]
changed: [server4]
changed: [server2]
changed: [server3]

TASK [Create test file] *****
changed: [server4]
changed: [server1]
changed: [server3]
changed: [server2]

PLAY RECAP *****

PLAY RECAP *****
server1      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server3      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server4      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp9 files$ ansible all -i inventory.ini -m comman
d -a "cat /root/ansible_test.txt"
server2 | CHANGED | rc=0 >>
Hello from Ansible
server4 | CHANGED | rc=0 >>
Hello from Ansible
server3 | CHANGED | rc=0 >>
Hello from Ansible
server1 | CHANGED | rc=0 >>
Hello from Ansible
```

containerization-and-devops

Experiment 10 C^D

##SONARQUBE

Step 1: Create Docker Network

```
PS C:\Users\VAANI JINDAL\OneDrive\Desktop\containerization\exp10files> wsl  
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files$ docker network create sonarqube-lab  
b877878e2ee330582f68caf31cbc1fa6e8d2feebc10a9b20a4ac430d088d6cb8
```

Step 2: Start PostgreSQL Database

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files$ docker run -d \
--name sonar-db \
--network sonarqube-lab \
-e POSTGRES_USER=sonar \
-e POSTGRES_PASSWORD=sonar \
-e POSTGRES_DB=sonarqube \
-v sonar-db-data:/var/lib/postgresql/data \
postgres:13  
Unable to find image 'postgres:13' locally  
13: Pulling from library/postgres  
d7ecded7702a: Pull complete  
80e8d95fdf53: Pull complete  
fcfd49225910: Pull complete  
947bdcac0ea3: Pull complete  
097f34b018c0: Pull complete  
4d2c7a7b3dba: Pull complete  
8cb000c698bc: Pull complete  
ba168ff08d42: Pull complete  
bad194149a82: Pull complete  
5f1cded65589: Pull complete  
7d0650bb210f: Pull complete  
23a39b1003e9: Pull complete  
21f7a31efcb7: Pull complete  
ae52c8ce1a35: Pull complete  
Digest: sha256:4689940c683801b4ab839ab3b0a0a3555a5fe425371422310944e89eca7d8068  
Status: Downloaded newer image for postgres:13  
a8101a3b1f7f5c75481d87e068470e4e07f9fdebc345ee12a5246513ef10f2fa
```

Step 3: Start SonarQube Server

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files$ docker run -d \
--name sonarqube \
--network sonarqube-lab \
-p 9000:9000 \
sonarqube:lts
Unable to find image 'sonarqube:lts' locally
lts: Pulling from library/sonarqube
60d98d907669: Pull complete
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
1df735f481ad: Pull complete
5d5a1fad7028: Pull complete
eb27e3a98da1: Pull complete
c7ad1fe61e07: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts
85f544c30b966de331fb54c1fbef5ae35d1d64240affd90e245692412c596459

```

Step 4: Generate Token (IMPORTANT)

Inside SonarQube UI:

1. Go to My Account → Security
2. Click Generate Token
3. Copy it

Step 5: Create Sample Project

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files$ mkdir -p sample-java-app/src/main/com/example
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files$ cd sample-java-app
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files/sample-java-app$ nano src/main/example/Calculator.java
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files/sample-java-app$ |

```

```

GNU nano 7.2 src/main/java/com/example/Calculator
public class Calculator {

    public int divide(int a, int b) {
        return a / b; // Bug: divide by zero
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100; // Code smell
        return result;
    }

    public String getUser(String userId) {
        return "SELECT * FROM users WHERE id = " + userId; // SQL injection
    }

    public String getName(String name) {
        return name.toUpperCase(); // Null pointer risk
    }
}

```

Step 6: Create Sonar Config File and Run Sonar Scanner (Docker way)

```

/Example/calculator.java
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files/sample-java-app$ nano sonar-project.pro
erties
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files/sample-java-app$ docker run --rm \
--network sonarqube-lab \
-v "$(pwd)":/usr/src \
-sonarsource/sonar-scanner-cli \
-Dsonar.host.url=http://sonarqube:9000 \
-Dsonar.login=squ_2d0fd6a7b40656b561a8d332fbec99a49580a905
04:30:33.835 INFO Scanner configuration file: /opt/sonar-scanner/conf/sonar-scanner.properties
04:30:33.841 INFO Project root configuration file: /usr/src/sonar-project.properties
04:30:33.866 INFO SonarScanner CLI 8.0.1.6346
04:30:33.875 INFO Linux 6.6.87.2-microsoft-standard-WSL2 amd64
04:30:35.277 INFO Communicating with SonarQube Server 9.9.8.100196
04:30:36.299 INFO Load global settings
04:30:36.398 INFO Load global settings (done) | time=101ms
04:30:36.401 INFO Server id: 147B411E-AZ2eq1w0sIdZddnqAnrY
port
04:30:47.069 INFO More about the report processing at http://sonarqube:9000/api/ce/task?id=AZ2e2wUEsIdZddnqAxkz
04:30:47.082 INFO Analysis total time: 8.517 s
04:30:47.084 INFO EXECUTION SUCCESS
04:30:47.085 INFO Total time: 13.255s
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp10files/sample-java-app$ |

```

Step 8: View Results

Go to:

👉 <http://localhost:9000>

⏮ ⏭ ↻ <http://localhost:9000/projects/create> 🔍 ☆ 🔊

Vidyamandir History Downloads New Tab mind map

✖ You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration ? 🔍 Search for projects...

How do you want to create your project?

Do you want to benefit from all of SonarQube's features (like repository import and Pull Request decoration)? Create your project from your favorite DevOps platform. First, you need to set up a DevOps platform configuration.

From Azure DevOps

[Set up global configuration](#)

From Bitbucket Server

[Set up global configuration](#)

From Bitbucket Cloud

[Set up global configuration](#)

From GitHub

[Set up global configuration](#)

From GitLab

[Set up global configuration](#)

Are you just testing or have an advanced use-case? [Create a project manually.](#)

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministration

Search for projects...A

Sample Java Appmain

Last analysis of this Branch had 1 warningApril 18, 2026 at 10:09 AMVersion 1.0

OverviewIssuesSecurity HotspotsMeasuresCodeActivity

Project SettingsProject Information

Failed

2 conditions failed

D

Reliability Rating on New Code is worse than A

B

Maintainability Rating on New Code is worse than A

Fix issues before they fail your Quality Gate with [SonarLint](#) in your IDE. Power up with connected mode!

1

New Bugs

0

New Vulnerabilities

0

New Security Hotspots

22min

Added Debt

3

New Code Smells

Reliability

D

Security

A

Security Review

A

Maintainability

B

My IssuesAll

Filters

Issues in new code

Type

Bug1

Vulnerability0

Code Smell4

Severity

Blocker0

Critical1

Major1

Minor3

Info0

Bulk Change

1 / 5 issues37min effort

src/main/java/com/example/Calculator.java

Move this file to a named package.

11 minutes ago

Code Smell

Minor

Open

Not assigned

10min effort

Comment

convention

Make sure this expression can't be zero before doing this division.

2 minutes ago

Bug

Critical

Open

Not assigned

5min effort

Comment

cert, cwe, denial-of-service

Immediately return this expression instead of assigning it to the temporary variable "x".

2 minutes ago

Code Smell

Minor

Open

Not assigned

2min effort

Comment

clumsy

Remove this useless assignment to local variable "x".

2 minutes ago

Code Smell

Major

Open

Not assigned

15min effort

Comment

cert, cwe, unused

Remove this unused "x" local variable.

2 minutes ago

Code Smell

Minor

Open

Not assigned

5min effort

Comment

unused

<https://vaanijindal223.github.io/containerization-and-devops/exp10.html>

4/4

containerization-and-devops

EXPERIMENT -11

PART A –

1. Run single container

Command :

```
docker run nginx
```

Runs only one container manually

No coordination between multiple containers

2. Run multi-container using Compose

Command :

```
docker compose up -d
```

Runs multiple containers together (WordPress + MySQL)

Works only on single machine, no scaling/self-healing

PART B –

Step 0: docker-compose.yml

```

image: mysql:5.7
container_name: wordpress_db
restart: always
environment:
  MYSQL_ROOT_PASSWORD: rootpass
  MYSQL_DATABASE: wordpress
  MYSQL_USER: wpuser
  MYSQL_PASSWORD: wppass
volumes:
  - db_data:/var/lib/mysql

wordpress:
  image: wordpress:latest
  container_name: wordpress_app
  depends_on:
    - db
  ports:
    - "8080:80"
  restart: always

volumes:
  db_data:

```

Task 1: Check Current State (No Swarm)

```

docker compose down -v
docker ps

```

```

vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker compose down -v
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
84873095fa1f   wordpress:latest                    "docker-entrypoint.s..." 52 seconds ago Up 46 seconds 80/tcp
wpstack_wordpress.3.zddcbifs105quqi9s8ldbmln2
01cc79b1ee5d   wordpress:latest                    "docker-entrypoint.s..." 52 seconds ago Up 46 seconds 80/tcp
wpstack_wordpress.2.r06g71w4dm3xq97bllxjfs8yi
e82f0f1cdddf   wordpress:latest                    "docker-entrypoint.s..." 52 seconds ago Up 46 seconds 80/tcp
wpstack_wordpress.1.jrzb74qxv2bnh6uwquk8ylzej
af81dbf6f447   mysql:5.7                           "docker-entrypoint.s..." 52 seconds ago Up 46 seconds 3306/tcp, 33060/tcp
wpstack_db.1.x46cka7kh0wub2c33mx0vwn9y
e8109433b2d2   jenkins/jenkins:lts                "/usr/bin/tini -- /u..." 2 weeks ago   Up 54 seconds 0.0.0.0:8080->8080/tcp, [
::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp jenkins
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker swarm init
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join
another one.
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ |

```

Task 2: Initialize Docker Swarm

```
docker swarm init
```

```
ui800zeepy3fbwr5f1tpxnbq8 * LAPTOP-CDFS1GM3 Ready Active Leader 28.2.2
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker stack deploy -c docker-compose.
yaml wpstack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
Creating service wpstack_db
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$
```

Task 3: Deploy as Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
Creating service wpstack_db
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
w69nhpglvwt3 wpstack_db replicated 1/1 mysql:5.7
rporwi2mq8e0 wpstack_wordpress replicated 0/1 wordpress:latest *:8080->80/tcp
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$
```

Task 4: Verify Deployment

```
docker service ls
```

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker service scale wpstack_wordpress
=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ |
```

Task 5: Access Application

Open in browser:

```
http://localhost:8080
```

Task 6: Scaling (MAIN FEATURE)


```
docker service scale wpstack_wordpress=3
```

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
w69nhpglvwt3 wpstack_db replicated 1/1 mysql:5.7
rporwi2mq8e0 wpstack_wordpress replicated 3/3 wordpress:latest *:8080->80/tcp
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
k7343279e7zj wpstack_wordpress.1 wordpress:latest LAPTOP-CDFS1GM3 Running Running 3 minutes ago
nuqellvmzfvx wpstack_wordpress.2 wordpress:latest LAPTOP-CDFS1GM3 Running Running 43 seconds ago
eig3kkod4gs4 wpstack_wordpress.3 wordpress:latest LAPTOP-CDFS1GM3 Running Running 43 seconds ago
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ |
```

Verify and check running :

```
docker service ls
docker service ps wpstack_wordpress
```

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker ps | grep wordpress
d5c942b619ee wordpress:latest "docker-entrypoint.s..." 4 minutes ago Up 4 minutes 80/tcp wpstack_wordpress.2.n
uqe1lvmzfvxrpwpstack_wordpress.2
05dd609058d6 wordpress:latest "docker-entrypoint.s..." 4 minutes ago Up 4 minutes 80/tcp wpstack_wordpress.3.e
ig3kkod4gs442j3of9e4ckfbwpstack_wordpress.3
46dedbf33755 wordpress:latest "docker-entrypoint.s..." 7 minutes ago Up 7 minutes 80/tcp wpstack_wordpress.1.k
7343279e7zjz1pkwlksjl1bswpstack_wordpress.1
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ |
```

Task 7: Self-Healing Test

Step 1: Find container

```
docker ps | grep wordpress
```

```
Error response from daemon: Cannot kill container: 7343279e7zjz1pkwlksjl1bs: No such container: 7343279e7zjz1pkwlksjl1bs
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker kill 46de
46de
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ |
```

Step 2: Kill one container

```
docker kill <container-id>
```

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp11files$ docker service ps wpstack_wordpress
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR
cws77iggbgj0 wpstack_wordpress.1 wordpress:latest LAPTOP-CDFS1GM3 Running Running 23 seconds ago
k7343279e7zj \_ wpstack_wordpress.1 wordpress:latest LAPTOP-CDFS1GM3 Shutdown Failed 29 seconds ago "task: non-zer
o exit (137)"
nuqellvmzfvx wpstack_wordpress.2 wordpress:latest LAPTOP-CDFS1GM3 Running Running 6 minutes ago
eig3kkod4gs4 wpstack_wordpress.3 wordpress:latest LAPTOP-CDFS1GM3 Running Running 6 minutes ago
```

Step 3: Check again

```
docker service ps wpstack_wordpress
```

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI_JINDAL/OneDrive/Desktop/containerization/exp11files$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI_JINDAL/OneDrive/Desktop/containerization/exp11files$
```

Task 8: Remove Stack

```
docker stack rm wpstack
```

PART C – ANALYSIS (WRITE IN EXAM)

Feature	Docker Compose	Docker Swarm
Scope	Single machine	Cluster (multi-node)
Scaling	Manual (--scale)	Built-in
Load Balancing	No	Yes
Self-Healing	No	Yes

containerization-and-devops

EXPERIMENT 12

PART A – CONCEPT (SHORT + WRITEABLE)

◆ Objective

- Understand Kubernetes basics
- Deploy application
- Scale application
- Perform self-healing

◆ Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run apps
Large ecosystem	Many tools/plugins
Cloud-native support	Works on AWS, GCP, Azure

PART B –

Step 1: Create file

```
GNU nano 7.2                               wordpress-deployment.yaml *
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

wordpress-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

Step 2: Apply deployment

```
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ kubectl apply -f wordpress-deployment.
yaml
deployment.apps/wordpress created
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ |
```

```
kubectl apply -f wordpress-deployment.yaml
```


Task: Scaling

```

Scaling connection for 3030
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ kubectl scale deployment wordpress --r
licas=4
deployment.apps/wordpress scaled
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-5vvdg          1/1     Running   0           10s
wordpress-6698dd7d66-6mv7j          1/1     Running   0           17m
wordpress-6698dd7d66-pcsk7          1/1     Running   0           10s
wordpress-6698dd7d66-twghh          1/1     Running   0           17m
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ |

```

```

kubectl scale deployment wordpress --replicas=4
kubectl get pods

```

TASK 5: self healing scaling

```

wordpress-6698dd7d66-twghh          1/1     Running   0           17m
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-5vvdg          1/1     Running   0           5m22s
wordpress-6698dd7d66-6mv7j          1/1     Running   0           22m
wordpress-6698dd7d66-pcsk7          1/1     Running   0           5m22s
wordpress-6698dd7d66-twghh          1/1     Running   0           22m
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ kubectl delete pod wordpress-6698dd7d6
6-5vvdg
pod "wordpress-6698dd7d66-5vvdg" deleted from default namespace
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-6mv7j          1/1     Running   0           24m
wordpress-6698dd7d66-nrxc5          1/1     Running   0           78s
wordpress-6698dd7d66-pcsk7          1/1     Running   0           7m14s
wordpress-6698dd7d66-twghh          1/1     Running   0           24m
vaani@LAPTOP-CDFS1GM3:/mnt/c/Users/VAANI JINDAL/OneDrive/Desktop/containerization/exp12files$ |

```

```

# Get pods
kubectl get pods

# Delete one pod
kubectl delete pod <pod-name>

# Check again
kubectl get pods

```

PART C – Swarm vs Kubernetes (Simple Comparison)

Feature	Docker Swarm	Kubernetes
Setup	Very easy	More complex
Scaling	Basic	Advanced (auto-scaling)

Feature	Docker Swarm	Kubernetes
Ecosystem	Small	Huge (monitoring, logging)
Industry use	Rare	Standard



Conclusion

- Kubernetes helps manage containerized applications efficiently
- It supports scaling and self-healing automatically
- It is widely used in industry compared to Docker Swarm